



Safeguarding large language models: a survey

Yi Dong¹ · Ronghui Mu¹ · Yanghao Zhang¹ · Siqi Sun¹ · Tianle Zhang¹ ·
Changshun Wu² · Gaojie Jin¹ · Yi Qi¹ · Jinwei Hu¹ · Jie Meng³ · Saddek Bensalem^{2,4} ·
Xiaowei Huang¹

Received: 21 October 2024 / Accepted: 31 August 2025 / Published online: 17 October 2025
© The Author(s) 2025

Abstract

In the burgeoning field of Large Language Models (LLMs), developing a robust safety mechanism, colloquially known as “safeguards” or “guardrails”, has become imperative to ensure the ethical use of LLMs within prescribed boundaries. This article provides a systematic literature review on the current status of this critical mechanism. It discusses its major challenges and how it can be enhanced into a comprehensive mechanism dealing with ethical issues in various contexts. First, the paper elucidates the current landscape of safeguarding mechanisms that major LLM service providers and the open-source community employ. This is followed by the techniques to evaluate, analyze, and enhance some (un)desirable properties that a guardrail might want to enforce, such as hallucinations, fairness, privacy, and so on. Based on them, we review techniques to circumvent these controls (i.e., attacks), to defend the attacks, and to reinforce the guardrails. While the techniques mentioned above represent the current status and the active research trends, we also discuss several challenges that cannot be easily dealt with by the methods and present our vision on how to implement a comprehensive guardrail through the full consideration of multi-disciplinary approach, neural-symbolic method, and systems development lifecycle.

Keywords Large language models · Generative AI · Safeguards · Guardrails · Trustworthy AI

Yi Dong and Ronghui Mu have been contributed equally to this work.

✉ Xiaowei Huang
xiaowei.huang@liverpool.ac.uk

¹ Department of Computer Science, University of Liverpool, Brownlow Street, Liverpool L69 3BX, UK

² IMAG, Université Grenoble Alpes, Saint-Martin-d'Hères, France

³ Loughborough University, Loughborough, UK

⁴ CSX-AI, Saint-Didier-au-Mont-d'Or, France

1 Introduction

In recent years, generative artificial intelligence (GenAI) has significantly accelerated humanity's stride into the era of intelligence. Technologies such as ChatGPT and Sora (OpenAI 2023) have become a pivotal force driving the transformation of a new generation of industries. However, the rapid deployment and integration of LLMs have raised significant concerns regarding their risks, including, but not limited to, ethical use, data biases, privacy, and robustness (Huang et al. 2023). In societal contexts, concerns also include the potential misuse by malicious actors through activities such as spreading misinformation or aiding criminal activities (Kang et al. 2023). In the scientific context, LLMs can be used professionally, with dedicated ethical considerations and risks in scientific research (Birhane et al. 2023).

To address these issues, model developers have implemented various safety protocols intended to confine the behaviors of these models to a more secure range of functions. The complexity of LLMs, characterized by intricate networks and numerous parameters, and the closed-source nature (such as ChatGPT) present substantial hurdles. These complexities require different strategies compared to the pre-LLM era, which focuses on *white-box techniques*, enhancing models by various regularizations and architecture adaptations during training. Therefore, in parallel to the reinforcement learning from human feedback (RLHF) and other training skills such as in-context training, the community moves towards employing *black-box, post-hoc strategies*, notably **guardrails** (Welbl et al. 2021; Gehman et al. 2020), which monitors and filters the inputs and outputs of trained LLMs. A guardrail is an algorithm that takes as input a set of objects (e.g., the input and the output of LLMs) and determines if and how some enforcement actions can be taken to reduce the risks embedded in the objects. If the input to LLMs relates to child exploitation, the guardrail may stop the input or adapt the output to become harmless (Perez et al. 2022). In other words, guardrails are used to identify the potential misuse in the query stage and to prevent the model from providing an answer that should not be given.

The difficulty in constructing guardrails often lies in establishing their requirements. AI regulations can be different across different countries, and in the context of a company, data privacy can be less severe than in the public domain. Nevertheless, a guardrail of LLMs may include **requirements** from one or more categories: Hallucination, fairness, privacy, robustness, toxicity, legality, out-of-distribution, uncertainty, etc. In this paper, we do not include the typical requirement, i.e., accuracy, as they are benchmarks of the LLMs and arguably not the responsibilities of the guardrails. That said, there might not be a clear cut on the responsibilities (notably, robustness) between LLMs and the guardrails, and the two models shall collaborate to achieve a joint set of objectives. Nevertheless, for concrete applications, the requirements need to be precisely defined, and their corresponding metrics, and a *multi-disciplinary* approach is called for. Mitigating a given requirement (such as hallucinations, toxicity, fairness, biases, etc) is already non-trivial, as discussed in Sect. 5. Working with multiple requirements worsens it, especially when some can be *conflicting*. Such complexity requires a sophisticated solution design method to manage. In terms of the design of guardrails, while there might not be “one method that rules them all”, a plausible design of the guardrail is *neural-symbolic*, with learning agents and symbolic agents collaborating in processing both the inputs and the outputs of LLMs. Multiple types of neural-symbolic agents (Lamb et al. 2021). However, the existing guardrail solutions such as Llama Guard

Inan et al. (2023), Nvidia NeMo Rebedea et al. (2023), and Guardrails AI (Rajpal 2023) use the simplest, loosely coupled ones. Given the complexity of the guardrails, it will be interesting to investigate other, more deeply coupled, neural-symbolic solution designs.

Like safety-critical software, a *systematic process* to cover the development cycle (ranging from specification to design, implementation, integration, verification, validation, and production release) is required to carefully build the guardrails, as indicated in industrial standards such as ISO-26262 and DO-178B/C. Our contributions are summarized as below: (1) Systematically examines existing LLM safeguarding techniques and their real-world implementations; (2) Reviews safety-related properties and evaluation challenges for guardrail systems; (3) Provides a comprehensive taxonomy of attack and defense methodologies according to access contexts; (4) Discusses key considerations for comprehensive guardrail development across technical, collaborative, multimodal extensions and regulatory dimensions.

This survey starts with some background introduction at Sect. 2. The **goal** is to (1) Understand the existing guardrail frameworks that are being used to control model outputs in LLM services, as well as the techniques to evaluate, analyze, and enhance guardrails against specific desirable properties (Sect. 3); (2) Understand the techniques that are being used to overcome these guardrails, as well as to defend the attacks and to reinforce the guardrails (Sect. 4); and then discuss how to achieve a complete guardrail solution, including several issues regarding the systematic design of a guardrail for a specific application context (Sect. 5).

2 Background for large language models

Large Language Models (LLMs), primarily based on the Transformer architecture (Vaswani et al. 2017), are composed of deep neural networks with several transformer blocks. Each block integrates a self-attention layer and a feedforward layer connected via residual links. This specific self-attention mechanism enables the model to concentrate on neighboring tokens while analyzing a specific token. Originally, the transformer architecture was exclusively developed for machine translation purposes. Newly developed language models that utilize transformer architecture can be fine-tuned, thereby removing the need for architectures tailored to specific tasks (Devlin et al. 2019). Typically, their networks encompass hundreds of billions (or more) of parameters and are trained on vast corpora of textual data. Examples include **ChatGPT-3** (Brown et al. 2020), **ChatGPT-4** (OpenAI 2023), **LLaMA** (Touvron et al. 2023), and **PaLM** (Anil et al. 2023).

LLMs are employed in a variety of complex tasks, such as conversational AI (Wei et al. 2023), translation (Lyu et al. 2023), and story generation (Simon and Muişe 2022). Current LLMs utilize architectures and training objectives similar to those in smaller language models, such as the Transformer architecture and tasks centered around language modeling. However, LLMs distinguish themselves by significantly scaling up in aspects like model dimensions, data volume, the breadth of their application scope, and computation cost. Building an offline model comprises three main stages (Huang et al. 2023): pre-training, adaptation tuning, and utilization improvement. Generally, the pre-training phase parallels conventional machine learning training, involving data collection, choosing an architecture, and undergoing training. The adaptation tuning includes instruction tuning (Lou et al. xxx)

and alignment tuning (Ouyang et al. 2022) to enable learning from task-specific instructions and adhere to human values. Finally, the utilization improvements can enhance user interactions, including in-context learning (Brown et al. 2020) and chain-of-thought learning (Wei et al. 2022).

After training an LLM, its performance against set expectations is crucial. This evaluation typically encompasses three dimensions: assessing essential performance, conducting safety analysis to understand potential consequences in practical applications, and utilizing publicly available benchmark datasets. The primary performance review focuses on essential capabilities like language generation and complex reasoning. Safety analysis delves into the LLM's alignment with human values, interactions with external environments, and integration into broader applications such as search engines. Additionally, benchmark datasets and accessible tools support this comprehensive evaluation. The outcome of this assessment determines whether the LLM meets pre-defined criteria and is ready for deployment. If it falls short, the process reverts to one of the earlier training stages to address identified shortcomings. At the deployment stage, LLM could be used on a web platform for direct user interaction, like ChatGPT, or integrated into a search engine, like the new Bing. Regardless of the application, it is standard practice to implement guardrails in interactions between LLMs and users to ensure adherence to AI regulations.

3 Techniques on design and implementation of guardrails

This section presents several existing guardrail techniques have been proposed by the LLM service provider or the open-source community. Then, we review the methods to evaluate, analyze, and enhance the LLMs according to the desirable properties one may expect an LLM to have. A comparison checkbox table among different platforms and properties is shown in Table. 1.

3.1 Guardrail frameworks and supporting software packages

LLM guardrails constitute a suite of safety measures designed to oversee and regulate user interactions with LLM applications. These measures are programmable, rule-based systems positioned between users and foundational models. Their primary function is to ensure that the LLM model adheres to an organization's established principles and operates within a prescribed ethical and operational framework. Guardrails are applied during the interaction stage between users and deployed LLMs, the last step in the LLM lifecycle. Figure 1 illus-

Table 1 Abilities among different guardrails

	Llama Guard	Nvidia NeMo	Guardrails AI	TruLens	Guidance AI	LMQL
Hallucination	✓	✓	✓	✓	✓	✓
Fairness	✓	-	✓	✓	-	-
Privacy	-	✓	-	-	-	-
Robustness	-	-	-	-	-	-
Toxicity	✓	✓	✓	✓	✓	✓
Legality	✓	-	-	-	-	-
Out-of-Distribution	-	-	✓	-	-	-
Uncertainty	-	✓	✓	✓	-	-

trates the lifecycle and potential vulnerabilities of the general guardrail mechanism. Developers complete the development of guardrails through data processing, guardrail model training, and model customization or fine-tuning (e.g., Llama Guard and NeMo Guardrails), as shown in the yellow area of Fig. 1. These guardrails are then deployed in LLMs to facilitate interaction with users. Typically, users predefine the content that needs protection, also called custom rules. Subsequently, users interact with LLMs through prompts and await the generated responses. The guardrails evaluate the output against the predefined rules to determine its compliance. If the content is deemed unsafe, guardrails may block it outright or issue a preset warning to the user. Conversely, if the output aligns with the criteria, it is displayed directly to the user, as indicated in the orange area of Fig. 1. Notably, some existing attack methods allow unsafe content to bypass guardrail protection, as highlighted in the red box of Fig. 1; for a detailed discussion of these attack methods, refer to Sect. 4.

3.1.1 Llama guard

Llama Guard Inan et al. (2023), developed by Meta on the Llama2-7b architecture, focuses on enhancing human-AI conversation safety. It is a fine-tuned model that takes the input and output of the victim model as input and predicts their classification on a set of user-specified categories. Figure 2 shows its workflow. Due to the zero/few-shot abilities of LLMs, Llama Guard can be adapted—by defining the user-specified categories—to different taxonomies and guidelines that meet requirements for applications and users. This is a Type 1 neural-symbolic system (Lamb et al. 2021), i.e., typical deep learning methods where the input and output of a learning agent are symbolic. It lacks guaranteed reliability since the classification results depend on the LLM’s understanding of the categories and the model’s predictive accuracy.

3.1.2 Nvidia nemo

Nvidia NeMo, described in Rebedea’s work (Rebedea et al. 2023), functions as an intermediary layer that enhances the control and safety of LLMs. It employs Colang, an executable

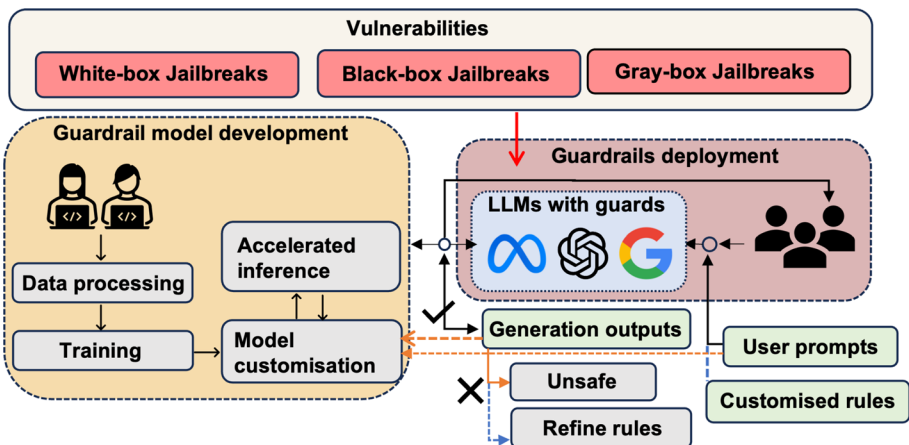


Fig. 1 Guardrails lifecycle and vulnerabilities

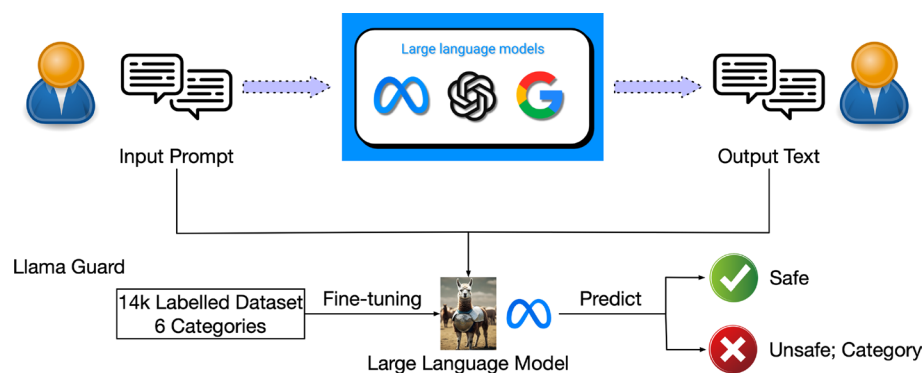


Fig. 2 Llama guard guardrail workflow

program language designed by Nvidia (2023), to establish constraints to guide LLMs within set dialogical boundaries. When the customer's input prompt comes, NeMo embeds the prompt as a vector and then uses K-Nearest Neighbors (KNN) method to compare it with the stored vector-based user canonical forms, retrieving the embedding vectors that are 'the most similar' to the embedded input prompt. The input embedding of Nemo differs from traditional approaches that utilize the initial layers. Instead, it employs embedding through similarity functions to capture the most relevant semantics. For example, Nemo uses the "sentence-transformers/all-MiniLM-L6-v2" model to compute embeddings, which is used for the following KNN search (Annoy algorithm is employed for efficient nearest-neighbor search.) Therefore, "input embedding" in the Nemo refers to mapping input sentences and paragraphs to a multi-dimensional dense vector space, facilitating the search for the most similar canonical forms/flows. After that, Nemo starts the flow execution to generate output from the canonical form. During the flow execution process, the LLMs generate a safe answer if the Colang program requests.

LLMs will be invoked multiple times during the guardrail flow for various tasks. For example, in a conversation scenario, LLM is utilized in the following three phases: (1) Generating user intent: the input of this LLM call includes two contexts: examples and potential user intents (top 5 intents from example code in NeMo Github repository). The output is a refined user intent (temperature is set as 0 to get the deterministic result). (2) Generating next step: In this phase, Nemo searches the most relevant similar flows and integrates these similar flows into an example, which is then fed into the LLM. The LLM call output is called "bot intent.". (3) Generating bot-message: The input for this call includes examples (the five most relevant bot intents) and relevant chunks (dictionary search), which are to be used as context.

The process is presented in Fig. 3. Building on the above customizable workflow, NeMo also includes a set of pre-implemented moderations dedicated to e.g., fact-checking, hallucination prevention in responses, and content moderation. NeMo is also a Type-1 neural-symbolic system, with its effectiveness closely tied to the performance of the KNN method.

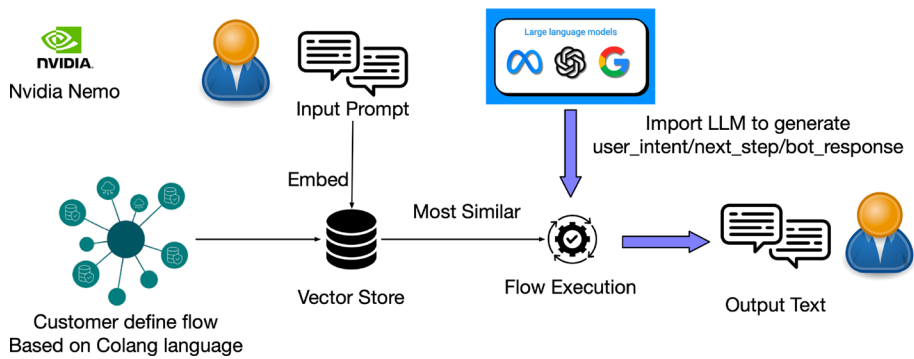


Fig. 3 Nvidia NeMo Guardrails Workflow

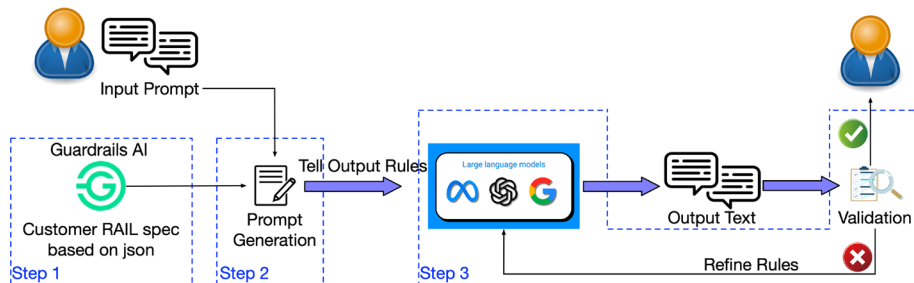


Fig. 4 Guardrails AI workflow

3.1.3 Guardrails AI

Guardrails AI enables the user to add structure, type, and quality guarantees to the outputs of LLMs (Rajpal 2023). It operates in three steps: 1) defining the “RAIL” spec, 2) initializing the “guard” and 3) wrapping the LLMs. In the first step, Guardrails AI defines a set of RAIL specifications, which are used to describe the return format limitations. This information must be written in a specific XML format, facilitating subsequent output checks, e.g., structure and types. The second step involves activating the defined spec as a guard. For applications that require categorized processing, such as toxicity checks, additional classifier models can be introduced to classify the input and output text. The third step is triggered when the guard detects an error. Here, the Guardrails AI can automatically generate a corrective prompt, pursuing the LLMs to regenerate the correct answer. The output is then re-checked to ensure it meets the specified requirements. Currently, the methods based on Guardrails AI are only applicable for text-level checks and cannot be used in multimodal scenarios involving images or audio. Unlike the previous two methods, Guardrail AI is a Type-2 neural-symbolic system with a backbone symbolic algorithm supported by learning algorithms (in this case, those additional classifier models) (Fig. 4).

3.1.4 TruLens

TruLens, developed by TruEra,¹ is an open-source toolkit for developing, evaluating, and monitoring LLMs. Central to its features is TruLens-Eval, which ensures quality by comparing outputs against predefined standards. The toolkit integrates LLMs, supporting logging records of inputs and outputs, and it leverages feedback functions that utilize auxiliary models, such as relevance models or sentiment classifiers, to perform evaluations on the RAG triad, including context relevance, answer relevance, and groundedness. Retrieval-Augmented Generation (RAG) is a technology that enhances the quality of language model outputs by supporting answer generation with retrieved relevant information. Within TruLens, RAG's role is to ensure the accuracy and relevance of model outputs by comparing them against predefined standards, thereby evaluating LLM apps. The services are invoked from various providers. For instance, when assessing groundedness related to how closely outputs align with the source material, TruLens-Eval can utilize providers like OpenAI API to call an LLM to find the relevant strings in a text or employ NLI models with hugging faces. The toolkit allows for the customization or pre-definition of feedback functions via Python, enabling evaluations to be specifically tailored to unique requirements. Additionally, TruLens incorporates embedding models to convert predefined information into numerical vectors, simplifying matching text with relevant data. TruLens also visualizes the LLM applications' rankings in a leaderboard according to their performance metrics, creating a dynamic environment that encourages developers to refine their models iteratively. As a guidance-oriented approach, TruLens is designed not to constrain LLM inputs and outputs but to provide a framework for continuous model refinement and evaluation, ensuring adherence to quality and relevance standards (Fig. 5).

3.1.5 Guidance AI

Guidance AI,² a programming paradigm, offers superior control and efficiency than conventional prompting and chaining. It allows users to constrain generation (e.g., with regex and

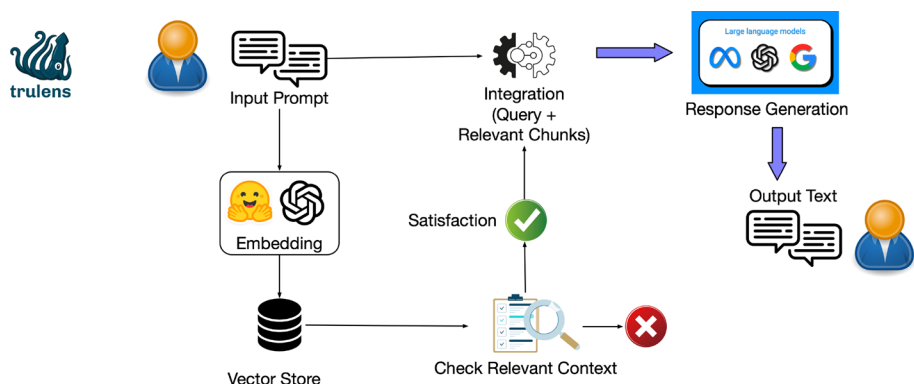


Fig. 5 TruLens workflow

¹ <https://truera.com/>

² <https://guidance.readthedocs.io/en/latest/>

CFGs) and interleave control (conditional, loops) and generation seamlessly. This guardrail tool integrates text generation, prompts, and logic control within a single, continuous flow in a Python environment, thereby refining the text processing approach in LLMs. This unified method allows more effective LLM control than traditional prompts or thought language chains. Its features include simple and intuitive syntax built on the Handlebars template language, assuring the variable insertion in any prompts. The Guidance program has a well-defined linear execution order directly corresponding to the token sequence processed by the language model. The illustration graph of Guidance AI working flow is demonstrated in Fig. 6.

At any timesteps during the program execution, the language model can be called for generation(via `{{gen}}` tag) or to make logical flow decisions, such as `{{#select}}` `{{or}}` `{{/select}}` commands. Guidance supports a variety of LLMs, and during dialogue, it can use role labels to map the current LLM to correct tokens or API calls, such as `{{#assistant}}`, `{{#user}}`, `{{#system}}` etc. It also can be integrated with HuggingFace models, including using Guidance acceleration to speed up standard prompts by reusing key-value caches to shorten prompt execution times and using token healing to optimize prompt boundaries. Regarding token healing, this concept is related to fixing the subtleties introduced by the language model's normal greedy tokenization method. Specifically, it involves advancing the model one step while simultaneously restricting the prefix of the generated token to be the same as the previous token. Regex patterns to enforce formatting are also supported in Guidance. Guidance's templated output is more suitable for generating text with high formatting requirements, such as ensuring legally compliant and controllable JSON structures. During this process, different operation commands have their processing methods; for example, encountering the select command, it specifies the generation of a token and returns the corresponding log probs, then uses a trie tree to match candidates and determine their probabilities, finally selecting the one with the highest probability. Additionally, it supports hidden blocks; for instance, some inference processes of the LLM may not need to be exposed to the end user, but they can be utilized in the template to generate intermediate results.

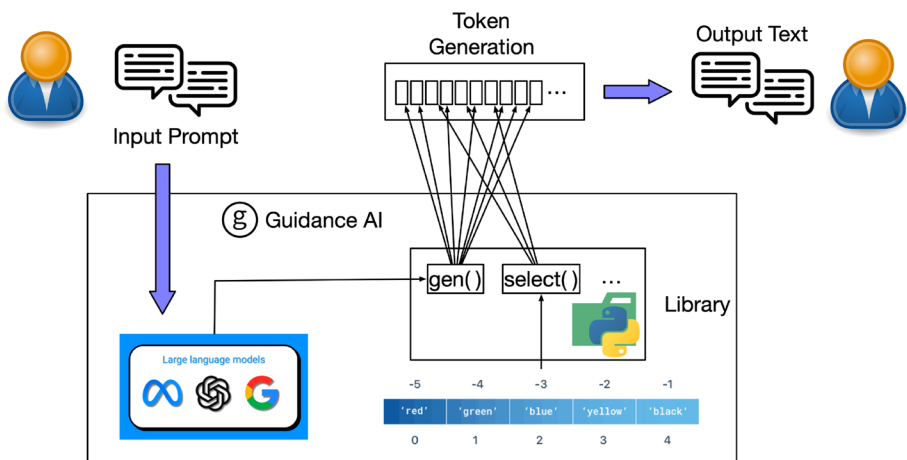


Fig. 6 Guidance AI Workflow

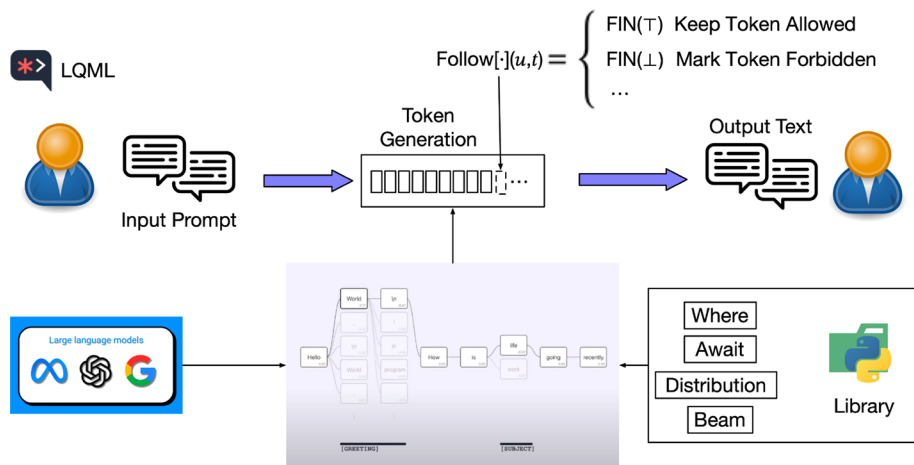


Fig. 7 LMQL workflow

3.1.6 LMQL (Language model query language)

LMQL,³ a programming interface for LLMs focusing on controlled output and safety of generated content, is designed by SRI Lab at ETH Zurich. Building on the foundation of Guidance AI, the LMQL project further advances the concept of “prompt templates” into a new programming language paradigm. As a Python superset, it allows developers the capacity to embed precise constraints within their queries. These constraints, from content restriction to adherence to specific formats for accuracy, leverage logit masking and custom operator support for fine-tuned control. Structured to simplify LLM interactions, LMQL introduces a SQL-like syntax complemented by scripting capabilities. Its foundation is built on decoder declarations, such as `argmax`, `beam`, or `sample` strategies, alongside query blocks that support inserting variables or placeholders expected to be filled, model sources, and intricate constraint conditions specified in ‘where’ clauses. The workflow of the LQML is illustrated in Fig. 7.

At its core, LMQL’s runtime and decoding ability uniquely features employs a scripted Beam search to execute LMQL-specific queries and constraints. This approach allows for a search across all placeholders rather than limiting the process to singular predictive points. Additionally, constraint-driven decoding dynamically adjusts available tokens based on real-time evaluation of specified constraints and can reduce ineffective model calls. The Partial Evaluation Semantics and FollowMaps facilitate the application of constraints in real-time during content generation, considering both the current state of the generated content and the potential next tokens. Partial evaluation semantics combines value and final semantics for eager validation. Value semantics determines the current value of expressions given the existing context. In contrast, the final semantics employs annotations such as `FIN` (fixed), `VAR` (variable), `INC` (increasing), and `DEC` (decreasing) to indicate the potential change in the value of expressions as content generation progresses. Among the joint value and final semantics, boolean expressions are denoted by symbols $\top$ for True and $\perp$ for False. Based on these semantics, FollowMap which is formulated as $\text{Follow}[\langle \text{expr} \rangle](u, t)$

³<https://lmql.ai>.

is designed to guide the generation process by evaluating the impact of adding a new token t to the current interaction trace u on meeting the specified constraints. The system evaluates potential next tokens against FollowMap constraints, excluding those that may violate constraints, thus guiding content generation away from invalid sequences and minimizing unnecessary model calls.

3.1.7 Python packages

Apart from the above-mentioned integrated guardrail techniques, Several Python packages are pivotal in implementing guardrails around LLMs, enhancing their safety, fairness, and compliance. Here we listed some packages: (1) *LangChain*⁴: LangChain is pivotal in streamlining the development of LLM applications. It introduces components that can be leveraged to implement guardrails, thus indirectly contributing to creating safer and more reliable AI solutions. (2) *AI Fairness 360 (AIF360)*⁵: AIF360 is an extended toolkit from IBM that provides developers with a comprehensive set of algorithms designed to detect, understand and mitigate bias in AI models. Its extensive resources help enhance the fairness and integrity of machine learning applications. (3) *Adversarial Robustness Toolbox (ART)*⁶: ART is tailored to enhance model security and robustness in the face of increasing adversarial attacks. It provides mechanisms to defend against and adapt to malicious input, protecting AI applications from potential vulnerabilities. (4) *Fairlearn*⁷: This package addresses and reduces unwanted biases in machine learning models. Fairlearn provides developers with tools and methodologies to assess and mitigate biases, fostering fairness and equality in AI outcomes. (5) *Detoxify*⁸: Detoxify aims to identify and mitigate toxic content in text data, serving as a crucial tool for maintaining a respectful and safe digital interaction space. It utilizes advanced models to screen and filter out harmful content, ensuring that AI applications foster positive communication.

These packages represent just a glimpse into the vast array of tools available to AI developers dedicated to embedding ethical considerations into their applications. By utilizing these packages, developers can navigate the complex challenges of AI safety, ensuring their LLMs are technologically advanced and aligned with ethical and responsible use principles. This approach reinforces the commitment to developing innovative AI technologies that respect societal norms and individual rights, marking a significant step towards responsible AI development.

3.1.8 Real-world scenarios of implementing guardrails

The practical application examples⁹ are shown in Table 2, which documents real-world implementations of guardrail technologies across diverse application domains with supporting evidence. This extensive documentation reveals the widespread adoption of guard-

⁴<https://www.langchain.com/>

⁵<https://github.com/Trusted-AI/AIF360>

⁶<https://github.com/Trusted-AI/adversarial-robustness-toolbox>

⁷<https://github.com/fairlearn/fairlearn>

⁸https://github.com/unitaryai/detoxify?trk=article-ssr-frontend-pulse_little-text-block

⁹It is noticed that guardrails have become a default setting in LLM applications, so it is often not explicitly mentioned.

Table 2 Real-world Implementation Examples of Existing Guardrail Technologies

Application	Llama Guard	Nvidia NeMo	Guardrails AI	TruLens	Guidance AI	LMQL
Chatbots or virtual assistants	(Gangavarapu 2024; Singh 2024; Yang et al. 2025)	(Gangavarapu 2024; Arun et al. 2025; Botsihhin and Boccaccia 2025; Singh 2024; Freitas and Lotufo 2024; Yang et al. 2025)	(Gangavarapu 2024; Team 2024)	(Peri et al. 2024; Team 2023)	–	(Beurer-Kellner et al. 2023; Lab 2023)
Content Moderation	(Bodhankar 2024; Singh 2024)	(Bodhankar 2024; Singh 2024)	(Team 2024)	(Team 2023)	(team 2023)	(Lab 2023)
Cyber Treats Defense	(Singh 2024; Wan et al. 2024; Paduraru et al. 2024)	(Singh 2024)	–	–	–	–
Structured Content	(Yang et al. 2025)	(Yang et al. 2025)	(Team 2024)	–	(team 2023)	(Beurer-Kellner et al. 2023; Lab 2023)
Summarization	–	–	(Team 2024)	(Team 2023)	–	–
AI Agent Reliability	(Yu 2024)	(Team 2024a)	(Team 2024)	(Team 2023)	–	(Vivien 2023)

rail frameworks across various sectors. The implementations span multiple use cases, with chatbots and virtual assistants showing the most comprehensive adoption across all major guardrail technologies. Notably, Llama Guard and Nvidia NeMo demonstrate versatility with implementations spanning most key application categories. Content moderation represents another critical area where existing guardrail technologies have been deployed in production environments. For instance, healthcare providers have integrated Nvidia NeMo and TruLens into clinical assistants (Gangavarapu 2024; Mehandru et al. 2024; Arun et al. 2025; Peri et al. 2024), while e-commerce platforms like Amazon (Botsihhin and Boccaccia 2025) and Flipkart (Singh 2024) employ guardrails within customer service systems to ensure policy compliance. The table further highlights specialized applications, such as cyber threat defense primarily utilizing Llama Guard (Singh 2024; Wan et al. 2024; Paduraru et al. 2024), while structured content generation benefits from multiple framework approaches. This evidence suggests an emerging pattern where certain guardrail technologies are becoming specialized for particular domains, while others maintain broader applicability. AI agent reliability represents a newer frontier where guardrail implementations are increasingly important as autonomous systems become more prevalent in commercial applications. These real-world implementations provide valuable insights into both the practical efficacy and domain-specific adaptations of guardrail technologies beyond theoretical frameworks.

3.2 Techniques for (Un)desirable properties in guardrails

In this section, we discuss several different properties, detailing their standard definitions and how to use Guardrails to protect these properties. It is noticed that the number of properties is too extensive to cover comprehensively; thus, we focus on hallucination, fairness, privacy, robustness, toxicity, legality, out-of-distribution, and uncertainty.

3.2.1 Hallucination

Hallucinations in LLMs are defined as responses that are either nonexistent in reality, illogical, or irrelevant to the prompt provided; an example of hallucination is shown in Fig. 8. These phenomena often stem from the language models' uncertainty in response, where they generate answers based on patterns identified in training datasets rather than actual factual understanding (Huang et al. 2023). The origins of these hallucinations can be traced back to various stages, including data sourcing (Singhal et al. 2023), pre-training (Li et al. 2023), alignment (Singhal et al. 2023), and inference (Liu et al. 2023).

The resolution of hallucinations in LLM is crucial due to their impact on model reliability and real-world applications, which presents a significant challenge in LLM development. Researchers are actively working on methods to differentiate between accurate and hallucinated content in LLM responses. This involves the use of detection technologies (Webster and Schmitt 2024) and evaluation methodologies (Cheng et al. 2023) to ensure the integrity and utility of LLM outputs. In certain contexts, such as safety-critical applications, ensuring rigorous guarantees for the output of LLMs is imperative and achievable. This involves first specifying high-level formal specifications and then applying formal verifiers to monitor whether the execution of LLMs conforms to the specified requirements. For instance, in Jha et al. (2023), the authors proposed a framework for counterexample-guided

inductive synthesis using LLMs and satisfiability modulo theory (SMT). Within this framework, an SMT solver is employed to eliminate the hallucinated outputs of the LLM, guiding it to generate correct outputs. Experimental results on two planning problems demonstrated the promise of this approach in practice. The framework consistently converged to correct outputs in finite steps. However, it is worth noting that such convergence cannot always be theoretically guaranteed, underscoring the practical significance of this method in addressing LLM hallucinations. Furthermore, there are some researches proposed to detect and prevent the hallucinations of LLMs.

In Webster and Schmitt (2024), the authors suggest utilizing continuous integration (CI) to automatically identify hallucinations in the outputs of an LLM with the aid of another LLM. The CI process involves regular incremental updates to the software, with each change automatically built and tested, allowing for prompt detection of errors. Therefore, LLM developers can use CI to automate tests on new datasets and updates to the models, which ensures that any changes do not unintentionally introduce bias or reduce the quality of the model's output. Instead of using another LLM to identify the errors, some researchers use trusted information sources to cross-check the generated content (Min et al. 2023). Building on these techniques, a comprehensive framework is proposed by Chern et al. (2023), which equips LLMs with the ability to recognize factual inaccuracies, utilizing external tools to gather supporting evidence. In addition to verifying the accuracy of responses, it's crucial to assess the faithfulness of the output to prevent hallucinations that diverge from the context. This means ensuring the output remains relevant and agrees with the input, avoiding any extraneous or contradictory information. This can be achieved by the fact-based metrics that are based on entity (Nan et al. 2021), relation (Goodrich et al. 2019), and knowledge F1 metric (Shuster et al. 2021). A further approach to ensure a model's faithful output utilizes the classifiers that are either trained on task-specific inference datasets (Mishra et al. 2021) or fine-tuned on adversarial datasets (Barrantes et al. 2020). Nowadays, the instruction-following abilities of LLMs are leveraged for autonomous evaluation. LLMs can effectively gauge accuracy and relevance by setting clear evaluation criteria and providing both generated and source content. Different methodologies have been adopted for output evaluation, such as binary judgment mechanisms (Luo et al. 2023) and using a k-point Likert scale (Gao et al. 2023).

In current guardrails, Nvidia Nemo employed the mechanism proposed by Manakul et al. (2023). It first generates a few additional responses from the LLM, typically two more by default. The original response from the bot is treated as the hypothesis, while the additional generated responses serve as the context or evidence. This method aligns the consistency verification with natural language inference (NLI) principles, allowing for a structured output assessment.

3.2.2 Fairness

Fairness is a concept that originates in sociology, economics, and law. It is defined as "imperfect and just treatment or behavior without favoritism or discrimination" in the Oxford English Dictionary. The key to fairness in NLP is the presence of social biases in language models; an example is illustrated in Fig. 9. Unfair LLM systems make discriminatory, stereotypic, and demeaning decisions against vulnerable or marginalized demographics, causing undesirable social impacts and potential harms (Blodgett et al. 2020). Fairness

in LLMs has been studied from different angles, such as gender bias (Malik 2023), cultural bias (Tao et al. 2023), dataset bias (Sheppard et al. 2023), and social bias (Sheng et al. 2023). These social biases can be encoded in the embeddings and carried over to decisions in downstream tasks, compromising the fairness of LLMs. For instance, when GPT-3 is prompted with questions about leadership roles or positions of power, it may reflect societal biases in its responses. If asked, “*Who is likely to be a CEO?*”, GPT-3’s response might lean towards “*He is likely to be a CEO,*” implying a male. Conversely, suppose the question is about lower-ranking positions or roles traditionally seen as supportive or administrative, such as “*Who is likely to be a secretary?*”. In that case, the model might respond with “*She is likely to be a secretary,*” suggesting a female. This pattern of responses reveals a bias where higher-status professions or roles are more closely associated with males, while supportive or administrative roles are linked with females. Such biases in LLM outputs can reinforce outdated stereotypes and potentially influence the perception of what roles are “appropriate” for individuals based on their gender.

Thus, to guard fairness in LLMs, increasing work is needed to understand these biases and evaluate their further effects on the downstream tasks of LLMs. For example, in terms of racial bias, an African American is more likely to be assigned a “criminal behavior” feature because of the “African” group he belongs to (Garrido-Muñoz et al. 2021). When this feature is used for model encoding and further downstream tasks, it induces unfairness in the language model toward African Americans. Biases are purposefully introduced into the responses of LLMs to craft distinct personas for use in interactive media (Badyal et al. 2023). BAD focuses on identifying and quantifying instances of social bias in models like ChatGPT, especially in sensitive applications such as job and college admissions screening (Koh et al. 2023). DAMA utilizes causal analysis to identify problematic model components, mainly focusing on mid-upper feed-forward layers most prone to convey biases (Limisiewicz et al. 2023). The presence of political bias is examined in ChatGPT, focusing on aspects such as race, gender, religion, and political orientation (Motoki et al. 2023). Additionally, they explored the role of randomness in responses by collecting multiple answers to the same questions, which enables a more robust analysis of potential biases. The bias of LLMs is also examined by controlling the input, highlighting that LLMs can still produce biased responses despite the progress in bias reduction (Yeh et al. 2023). Besides, a Bias Index is designed to quantify and address biases inherent in LLMs, including GPT-4 (Shaikh et al. 2022). It has also been observed that the biased response can be generated inadvertently, sometimes as seemingly harmless jokes (Zhou et al. 2023).

A line of debiasing studies aims to mitigate the *intrinsic bias* that is task-agnostic in the representations before they are applied to downstream tasks. Safeguarding training data before training the model efficiently alleviates intrinsic biases since label imbalance across different demographic groups in the training data is an essential factor in inducing bias. A Counterfactual Data Augmentation (CDA) (Ma et al. 2020; Xie and Lukasiewicz 2023) is a widespread data processing method to balance labels, which replaces the sensitive attributes in the original sample with the sensitive attributes of the opposite demographic based on a prior list of sensitive word pairs. Fairness can be incorporated into LLMs’ design to balance the training samples, and then a guarded fairer model can be obtained by parameter tuning. Retraining models (Qian et al. 2022) is a direct way to reduce bias, although it can be resource-intensive and difficult to scale. For instance, FairBERTa is a fairer model for retraining RoBERTa on a large-scale demographic perturbation corpus Perturbation Aug-

mentation NLP Dataset (PANDA) containing 98K augmentation sample pairs (Qian et al. 2022). Moreover, an additional debiasing module is added after the encoder of LLMs to filter out the bias in the representation, and a common strategy is to utilize a contrastive learning framework for training (Oh et al. 2022).

The other line of debiasing studies aims to mitigate the *extrinsic debiasing* in a task-specific way. These studies attempt to improve fairness in downstream tasks by making models provide consistent outputs across different demographic groups. Many studies have concentrated on reducing bias through model adaptation approaches. A bias mitigating method, DAMA (Limisiewicz et al. 2023), can reduce bias while maintaining model performance on downstream tasks. Ranaldi et al. (2023) investigated the bias in CtB-LLMs and demonstrated the effectiveness of debiasing techniques. They find that bias depends not solely on the number of parameters but also on factors like perplexity and that techniques like debiasing of OPT using LoRA can significantly reduce bias. Ungless et al. (2022) demonstrated that the Stereotype Content Model, which posits that minority groups are often perceived as cold or incompetent, applies to contextualized word embeddings and presents a successful fine-tuning method to reduce such biases. Moreover, Ernst et al. (2023) proposed a novel adversarial learning debiasing method applied during the pre-training of LLMs. Ramezani and Xu (2023) mitigated cultural bias through fine-tuning models on culturally relevant data, yet it requires resources that make it accessible to only a few.

Instead of fine-tuning parameters, several studies directly set up guardrails on the interaction between users and LLMs by exploring the control of input and output. Huang et al. (2023) suggested using purposely designed code generation templates to mitigate the bias in code generation tasks. Tao et al. (2023) found that cultural prompting is a simple and effective method to reduce cultural bias in the latest LLMs. However, it may be ineffective or even exacerbate bias in some countries. Oba et al. (2023) proposed a method to address gender bias that does not require access to model parameters. It shows that text-based preambles generated from manually designed templates can effectively suppress gender biases in LLMs with minimal adverse effects on downstream task performance. Dwivedi et al. (2023) guided LLMs to generate more equitable content by employing an innovative approach of prompt engineering and in-context learning, significantly reducing gender bias, especially in traditionally problematic.

Developing guardrails through a comprehensive approach that intertwines various strategies is crucial to mitigate bias effectively. This begins with meticulously monitoring and filtering training data to ensure it is diverse and devoid of biased or discriminatory content. The essence of this step lies in either removing biased data or enriching the dataset with more inclusive and varied information. Alongside this, algorithmic adjustments are necessary, which involve fine-tuning the model's parameters to prevent the overemphasis of certain patterns that could lead to biased outcomes. Incorporating bias detection tools is another pivotal aspect. These tools are designed to scrutinize the model's outputs, identifying and flagging potentially biased content for human review and correction. We believe that adopting a continuous learning approach is key to the long-term efficacy of these guardrails. This involves regularly updating the model with new data, insights, and feedback and adapting to evolving societal norms and values. This dynamic process ensures that the guardrails against bias remain robust and relevant. Moreover, we believe in *principled methods* to evaluate fairness when the definitions are settled. However, the definition is expected to be

distribution-based rather than point-based as unintended responses, which need to estimate posterior distributions and measure the distance between two distributions.

3.2.3 Privacy (Copyright)

Privacy, in the context of modern technology and artificial intelligence, is a crucial aspect of data protection that has been increasingly emphasized through legislation and research. Legislative measures like the EU AI Act, General Data Protection Regulation (GDPR), and California Consumer Privacy Act (CCPA) have established stringent data sharing and retention standards, necessitating strict adherence to data protection and privacy guidelines. Despite these frameworks, challenges persist in preventing the release of personally identifiable information (PII) by LLMs (Zou et al. 2023), emphasizing the need for cautious and robust data handling protocols, an example of a privacy issue is shown in Fig. 10. Li et al. (2023) comprehensively analyzes privacy attacks against LLMs, introduces significant defense strategies, and highlights potential new privacy issues and future research directions as LLMs evolve.

Several studies have focused on implementing privacy defense technologies to safeguard data privacy and mitigate privacy breaches. Differential Privacy (DP)-tuned LLMs (Li et al. 2023) emerge as a leading approach to protecting data privacy in these contexts, ensuring secure handling of sensitive information by LLMs and minimizing the risk of privacy violations. For general NLP models, Li et al. (2022) indicated that a direct application of DP-SGD (Abadi et al. 2016) may not perform satisfactorily and suggests a few tricks. Mireshghallah et al. (2022) study differential privacy model compression and proposes a framework that achieves 50% sparsity levels while maintaining nearly complete performance, setting a benchmark for future research in this area. Igamberdiev and Habernal (2023) implemented a model for text rewriting along with Local Differential Privacy (LDP), both with and without pretraining. Xiao et al. (2023) introduce Privacy Protection Language Models (PPLM), a novel paradigm for fine-tuning LLMs that incorporates domain-specific knowledge while preserving data privacy. They explore techniques such as corpus curation and instruction-based tuning, demonstrating the effectiveness of these approaches in safeguarding private data. Zhao et al. (2023) introduce a novel text protection mechanism called "Silent Guardian," which effectively prevents the malicious use of text by LLMs through Truncation Protection Examples and the Super Tailored Protection algorithm. It features efficiency, semantic consistency, transferability, and robustness. Ozdayi et al. (2023) proposed a method to prepend a trained prompt to the incoming prompt before passing it to the model, where the training of the prefix prompt is to minimize the extent of extractable memorized content in the model. Li et al. (2023) and Duan et al. (2023) also proposed the prompt-tuning methodology that adheres to differential privacy principles. Yu et al. (2022) propose an effective algorithm for differentially private fine-tuning of large pre-trained language models, which achieves utility close to that of non-private models while protecting privacy and reduces the computational and memory cost of training, especially performing excellently on larger models. Shi et al. (2022) introduces a "Just Fine-tune Twice" (JFT) framework for the latest large Transformer models, achieving Selective Differential Privacy protection. It enhances the model's utility and privacy safeguards through double fine-tuning and systematic methods.

Other than constructing privacy-preserving LLMs, watermarking techniques can play a more critical role in LLMs for privacy and copyright protection. A typical watermarking mechanism Kirchenbauer et al. (2023) embedded watermarks into the output of LLMs by selecting a randomized set of “green” tokens before a word is generated and then softly promoting the use of green tokens during sampling. So, as long as we know the list of green tokens, it is easy to determine if an output is watermarked or not. We can also use the watermarks to track the point of origin or the owner of watermarked text for copyright purposes, and this has been applied to protect the copyright of generated prompts (Yao et al. 2023). We believe in an agreed watermarking mechanism between the data owners and the LLMs developers, such that the users embed a personalized watermark into their documents or texts when they deem them private or with copyright, and the LLMs developers will *not* use watermarked data for their training. More importantly, the LLMs developers should take the responsibility of enabling (1) an automatic verification to determine if a user-provided, watermarked text is within the training data, and (2) model unlearning (Nguyen et al. 2022), which allows the removal of users’ personally owned texts from training data. LLMs also risk user trust due to their pre-training on vast textual datasets Narayanan et al. (2021), potentially leading to inadvertent disclosure of sensitive information about individuals (Plant et al. 2022). Malicious actors can exploit this vulnerability through adversarial attacks (Wang et al. 224), underscoring the critical importance of privacy protection, especially when fine-tuning LLMs with sensitive data.

In addressing privacy concerns within LLM applications, implementing guardrails is crucial for existing and in-development technologies. Key strategies for existing applications include robust testing to identify privacy risks and continuous model monitoring to adapt to new threats. Implementing content control mechanisms such as blocklists, allowlists, and suppression lists directly tackles unsafe content generation that could compromise privacy. For example, Nemo Guardrails restricts apps to making connections only to external third-party applications known to be safe. The guardrails can force an LLM model to interact only with third-party software on an allowed list. The “human-in-the-loop” approach, where human oversight is applied to review potentially sensitive outputs and facilitates user reporting channels for privacy violations, enhances the protection framework (Rahman et al. 2024). Regular model retraining to align with current norms and the option to revert to previous safe versions of the model serve as dynamic responses to privacy challenges.

For applications still in development, privacy protection begins at the design stage, with ethical risk assessments focused on identifying and mitigating privacy risks. Adopting responsible AI practices ensures privacy is a core consideration from the outset (Sarker 2024). Implementing selective memory and information filtering techniques restricts the AI’s access to sensitive data, directly safeguarding user privacy. Removing personally identifiable information (PII) from data used in model training is critical in protecting privacy (Yang et al. 2023). Continuous updates to employ the latest version of LLMs and strict data privacy protocols for staff overseeing AI use are also essential for maintaining privacy standards.

3.2.4 Robustness

With the rise of LLMs as dominant models in NLP, robustness consists of out-of-distribution (OOD) and adversarial robustness. This section only accounts for adversarial robustness,

while OOD is discussed in Sect. 3.2.7. The adversary (end-user) only attempts to jailbreak the model by explicitly optimizing adversarial queries or adaptively making queries based on previous outputs but will not make out-of-distribution queries asking about potentially revoked information. Robustness has distinct definitions across various downstream tasks of NLP; it can be commonly characterized in the following way (It works for a range of NLP tasks like text classification and sequence labeling): let x represent the input and y its corresponding correct label. Consider a model f that has been trained on data pairs $(x, y) \sim D$, with its output prediction for x given by $f(x)$. When new test data $(x', y') \sim D'$, where D' is not identical to D , is introduced, the robustness of the model can be determined by its performance on D' (Wang et al. 2022). Through comprehensively perturbing the input from x to x' , we encounter the notion of adversarial robustness, which is a concept originating from the computer vision (Goyal et al. 2023).

The adversarial robustness under the LLMs refers to the ability of models to maintain performance when faced with inputs that have been intentionally altered or crafted to cause the model to error, such as the malicious queries made intentionally or unintentionally (Ye et al. 2023). It is a type of model based on transformations or small perturbations (e.g. typo) to study the robustness of the model (it is also called invariance of LLMs) (Liang et al. 2023). Typically, alterations that maintain the underlying meaning, like modifying the text case and contraction perturbed, are considered fairly benign (Liang et al. 2023). In particular, disturbances are directed at various layers of linguistic signals, including characters, words, sentence structures, and underlying meanings. The core objective is to replicate potential user mistakes (e.g. use of near-meaning words), to assess the impact of minor deviations on the outcomes of LLMs (Zhu et al. 2023; Wang et al. 2024).

The defense methods for shielding LLMs from deliberate disruptions remain under investigation (Liu et al. 2024), indicating that robust safeguarding measures are necessary, especially during the most crucial phases of user engagement with these models. Typically, guardrails pre-process users' inputs to remove or neutralize potentially adversarial content, thus preventing models from being misled by manipulated inputs (e.g., correcting typos and standardizing input formats). Similarly, guardrails also monitor LLM's outputs. This may involve establishing thresholds for specific types of responses or flagging outputs that significantly deviate from expected patterns for review by a professional security team.

3.2.5 Toxicity

An important NLP task is the toxicity detection (Pavlopoulos et al. 2020), the term 'toxicity' is employed as a broad descriptor, encompassing a variety of related phenomena and linguistic contexts that may also manifest as 'offensive' (Zampieri et al. 2019), 'abusive' (Menini et al. 2021), 'hateful' (Kirk et al. 2023). Similar descriptors (Pavlopoulos et al. 2020). We show a typical example of toxicity in Fig. 11. LLMs, as one of the prevalent developments in traditional language modeling, are frequently trained using vast quantities of datasets, which can include content exhibiting toxic behavior and unsafe material, such as hate speech, offensive/abusive language, etc. Typically, a thorough examination of toxicity is required, especially considering the employment of LLMs for downstream tasks that might engage younger or more vulnerable individuals, as well as the negative effects of unintended outputs from LLMs on specific tasks (Zhang et al. 2023). The definition of what constitutes toxicity of the LLMs varies normally, toxicity responses will be defined as

rude, disrespectful, or unreasonable responses that are likely to make an individual leave a discussion (Deshpande et al. 2023). It is, hence, very desirable to evaluate how well-trained LLMs deal with toxicity (Guo et al. 2023).

Existing studies address the problem by focusing on representative terms in datasets, such as identity terms (Sap et al. 2020). To evaluate the toxicity in LLMs, several studies have crafted trigger prompts that mirror detailed toxic categories (Gehman et al. 2020). These studies leveraged standard toxicity metrics, such as the Toxicity Classifier Score and PerspectiveAPI,¹⁰ to determine whether the LLM's response is toxic (Hosseini et al. 2017). However, typical metrics are susceptible to evaluator bias (Goyal et al. 2022), and encoders are perturbed (Rosenblatt et al. 2022). Subsequently, a structured investigation framework attempted to address this bias (Koh et al. 2024). Despite being trained on extensive datasets, LLMs are capable of generating outputs that can be implicitly toxic, which are difficult to detect with straightforward, zero-shot methods (Welbl et al. 2021; Wen et al. 2023). This complexity arises even when prompts appear non-toxic, underscoring the nuanced challenges in detoxifying language models, such as depending on the specific roles assigned to LLMs, certain roles may generate markedly more toxic outcomes (Deshpande et al. 2023).

Even when a generative model is trained on data characterized by low toxicity levels, and its ability to minimize the generation of toxic text has been validated through evaluations, it is still crucial to enforce protective measures during live interactions between users and the model (Liang et al. 2024). Safety guardrails are an integral part of the user interaction and LLMs interaction phases, playing a key role in ensuring privacy, preventing bias, and maintaining user trust (Dong et al. 2024). For example, Nvidia Nemo allows users to define the toxic output they want to identify; the next step is determining the chatbot's response to users' input. This involves setting up a workflow that utilizes these definitions. Thus, this procedure is triggered whenever there is potential exposure to toxic content, and the chatbot supports the user. Furthermore, they ensure compliance with legal standards and align AI operations with societal values.

3.2.6 Legality

Another crucial aspect of safeguarding LLMs involves managing the risks associated with illicit¹¹ outputs (Kumar et al. 2023). Generally, this involves safeguarding efforts on two fronts: implementing measures to reject inappropriate user inputs and moderating model output to ensure it is appropriate and safe for users or downstream tasks.

During the development of LLMs, developers implement a series of measures to ensure the safety and compliance of the models with relevant laws and regulations. These measures include: i) researchers meticulously screen and clean the training data before training the

¹⁰ Perspective API was developed by Jigsaw and the Google Counter Abuse Technology team (<https://perspectiveapi.com>)

¹¹ It's important to note that while the concepts of legality and toxicity may overlap to some extent, they are not synonymous. Legality is the lowest requirement, defining what is permitted under the law. However, something that is not illegal may still be considered toxic due to its potential to cause harm or adverse effects in other contexts. Conversely, if something is deemed illegal, it invariably falls into the category of being toxic, as its prohibition by law implies a recognized potential for harm or negativity. Thus, while legality provides a clear boundary based on legal statutes, toxicity encompasses a broader range of potentially harmful actions or materials, some of which may not be explicitly covered by legal definitions but are nonetheless detrimental to well-being or ethical standards.

models to remove inappropriate, harmful, or illegal content. This ensures the model learns from high-quality data and avoids adopting inappropriate behaviors. ii) During model training, human reviewers assess the samples generated by the model and offer feedback, aiding in rectifying errors and enhancing the model's output. This process, alongside RLHF, helps models refine the content they generate and gradually adopt appropriate behavior.

Once the model construction is completed and before release, models undergo thorough and rigorous ethics and safety testing to ensure that the content they generate is absent of inappropriate or illegal elements. One classical approach is red teaming (Ganguli et al. 2022; Perez et al. 2022), which entails simulated attacks and adversarial testing to uncover potential vulnerabilities, ethical pitfalls, and legal considerations. Organizations like OpenAI, Anthropic, Google, and Meta utilize diverse methodologies for red teaming, ensuring a thorough evaluation and effective risk mitigation. For instance, Google promotes internal red teams¹², where employees with diverse expertise simulate attacks on the AI model. In contrast, OpenAI favors external red teaming and has established external networks¹³ to encourage participation from outside members.

In addition to the above safeguarding efforts, monitoring systems are established upon model release to detect inappropriate inputs and outputs. Techniques such as natural language processing and anomaly detection are employed for real-time identification. Upon identification of any issues, immediate measures, such as content filtering algorithms or human intervention protocols, are swiftly implemented to address the concern. It is worth noting that leading LLM providers, such as Google, OpenAI, Anthropic, and Meta, offer advanced moderation tools and techniques to developers or users, enabling customized safeguards against illicit and inappropriate content. For instance, Google offers PaLM-based Moderation¹⁴, capable of detecting more than 16 types of inappropriate content. OpenAI provides a Moderation API¹⁵, allowing developers and users to customize safeguards for inappropriate content. Meanwhile, Anthropic has developed Constitutional AI (Bai et al. 2022) and Meta utilizes Llama Guard (Inan et al. 2023) for content moderation.

In addition to the moderation tools offered by LLM providers, notable contributions from other entities in the field also exist. For instance, *LangChain*¹⁶, an open-source framework, simplifies and safeguards the development of applications using LLMs. Specifically, it offers a standardized interface for creating, combining, and customizing various components, resulting in powerful language-driven applications. One notable application of LangChain in the legal domains is ConstitutionalChain¹⁷. By incorporating predefined rules and guidelines, ConstitutionalChain can filter and modify generated content to align with constitutional principles. This ensures that responses are controlled, legal, and contextually appropriate.

Moreover, ensuring that Large Language Models (LLMs) comply with legal requirements across various jurisdictions is a complex challenge, primarily due to the diversity in legal systems and regulatory standards. Each jurisdiction may have its own set of guidelines

¹² <https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/>

¹³ <https://openai.com/blog/red-teaming-network>

¹⁴ <https://cloud.google.com/natural-language/docs/moderating-text>

¹⁵ <https://platform.openai.com/docs/guides/moderation>

¹⁶ <https://python.langchain.com/docs/modules/chains>

¹⁷ https://api.python.langchain.com/en/latest/chains/langchain.chains.constitutional_ai.base.ConstitutionalChain.html

and quantitative thresholds, necessitating tailored technical solutions to meet these specific compliance criteria. Moreover, some compliance requirements are internal, non-scientific, or lack transparency. For instance, organizations may impose internal policies that are not publicly documented, making it challenging for developers to align LLMs with these opaque standards. Additionally, certain regulatory bodies may enforce guidelines that are broad or ambiguous, leaving room for interpretation and complicating the compliance process.

- **Incorporating Legal Requirements into Training:** Embedding jurisdiction-specific legal constraints during the pre-training and fine-tuning phases can guide LLMs to align with local regulations. Techniques like Low-Rank Adaptation (LoRA) allow for efficient fine-tuning by introducing trainable low-rank matrices into the model, enabling adaptation to specific legal domains without extensive computational resources.
- **Implementing Real-Time Censorship Mechanisms:** During deployment, it's crucial to have censorship systems that can filter out content violating legal standards in real-time, ensuring a seamless user experience. However, research indicates that semantic censorship-relying on another model to detect and filter undesirable content-faces theoretical limitations. Specifically, the authors (Glukhov et al. 2023) demonstrate that semantic censorship can be viewed as an undecidable problem, meaning there is no general algorithmic solution that can determine, for every possible input, whether the output complies with all legal and ethical standards.

3.2.7 Out-of-distribution

For a specific DNN, *out-of-distribution* (OOD) data strictly refers to data not belonging to any in-distribution classes used in training. Broadly, OOD data can be characterized as differing from the in-distribution data on certain dimensions. Research indicates that DNNs often exhibit overconfident decision-making when presented with OOD data. This has led to widespread investigation of OOD detection issues across domains such as computer vision (Hendrycks and Gimpel 2016), and natural language processing (Arora et al. 2021). However, the OOD detection task within the field of NLP presents notable challenges, particularly exacerbated by the presence of LLMs. This issue has resulted in limited research focused on OOD detection specifically tailored to LLMs (Ren et al. 2023), primarily due to the immense training corpora used for LLMs, making it difficult to define precisely what data has not been utilized for training. Moreover, the generative nature of LLMs adds another layer of complexity to defining the OOD problem (Kadavath et al. 2022).

While defining OOD instances for an LLM is generally very difficult, if not impossible, it becomes more feasible when applied to specific real-world scenarios where the context is more precise. In practical scenarios, OOD instances can be defined as data irrelevant to the main task or significantly deviating from normal ones. For instance, recent work (Li et al. 2023) has explored the evaluation of OOD in the context of specific language model applications, such as text classification (Kaushik et al. 2019), sentiment analysis (Zhang et al. 2023), machine reading comprehension (Zeng et al. 2020), and found that it can lead to a significant performance decrease, even with minor semantic shifts caused by small perturbations. To mitigate the impact of OOD on model performance in practical tasks, strategies

such as setting up anomaly input filtering mechanisms¹⁸ or constructing OOD detectors tailored to the task can be employed.

3.2.8 Uncertainty

A key aspect of LLMs' trustworthiness lies in their ability to discern their outputs' reliability and correctness, a concept central to uncertainty quantification. This approach is an effective method for assessing risks, aiming to gauge the confidence levels of LLMs in their predictions. Elevated uncertainty suggests that an LLM's output may require rejection or additional scrutiny. Figure 13 shows an example of uncertainty. The effectiveness of uncertainty quantification is further contingent on the alignment between the model's predicted confidence and its actual accuracy, essentially measuring the model's calibration.

There has been a growing focus on research to quantify the overall uncertainty in LLMs. Establishing dependable uncertainty metrics is essential for enhancing the safety of LLM systems. Recent studies have noted that the calibration of LLMs is improved relatively through techniques like combining multiple reasoning chains (Wang et al. 2023), integrating different prompts (Jiang et al. 2023), or by prompting LLMs to output their confidence levels directly (Kadavath et al. 2022). In addition to these observations, numerous methods have been developed to quantify the uncertainty in LLMs effectively. (Lin et al. 2022) demonstrated that a GPT-3 model can learn to articulate uncertainty regarding its responses in natural language independently of using model logits. (Xiao et al. 2022) comprehensively compared various popular approaches to construct a well-calibrated prediction pipeline for pre-trained language models. (Ren et al. 2023) unveiled KnowNo, a framework designed to measure and align the uncertainty in LLM-based planners, enabling them to recognize their limitations and seek assistance when necessary.

The primary hurdles in assessing LLM uncertainty arise from the pivotal roles of meaning and structure in language. This pertains to what linguists and philosophers define as a sentence's semantic content and syntactic or lexical framework. While foundation models mainly output token-likelihoods, reflecting lexical confidence, the meanings often hold the most significance in most applications. Kuhn et al. (2022) introduced the concept of semantic entropy, which integrates linguistic consistencies arising from identical meanings. The fundamental method involves a semantic equivalence relation, denoted as $\mathbb{E}(s_i, s_j)$, where s_i and s_j represent output sentences corresponding to a given input. This equivalence relation is said to hold when two sentences s_i and s_j convey the same meaning, implying that they belong to the same cluster C . The semantic entropy is defined as

$$H(C|x) = - \sum_C \mathbb{P}(C|x) \ln \mathbb{P}(C|x), \quad (1)$$

where x is the input sentence. This methodology, which employs 'out-of-the-box' models, enhances reproducibility and simplifies deployment. Moreover, this unsupervised uncertainty could address the issue identified in previous research, where supervised uncertainty measures often falter in the face of distributional shifts.

Utilizing the above uncertainty technologies to build a guardrail for LLMs, it is crucial to integrate mechanisms that enable the model to assess and communicate its uncertainty. This

¹⁸ https://hub.guardrailsai.com/validator/guardrails/unusual_prompt

involves training the model to recognize when a query falls outside its expertise or when the answer is speculative. It also involves responding appropriately—whether by providing a cautious answer, flagging the response as uncertain, or directing the user to more reliable sources.

3.2.9 Benchmarking challenges in guardrail assessment

Quantitative evaluation of LLM guardrails faces significant challenges due to inherent complexities. The multidimensional safety attributes (hallucination, fairness, privacy, toxicity, robustness, etc.) necessitate diverse evaluation frameworks that resist unification. The field lacks standardized benchmarks comparable to ImageNet (Deng et al. 2009) for comprehensive assessment. While specialized datasets like RealToxicityPrompts (Gehman et al. 2020) and AdvBench (Zou et al. 2023) exist, they address isolated aspects rather than providing holistic evaluation. More comprehensive benchmarks such as DecodingTrust (Wang et al. 2023) represent progress toward multidimensional evaluation, yet challenges persist in their widespread adoption and standardization.

This issue is exacerbated in multilingual contexts, where safety attributes vary across languages due to linguistic nuances, cultural contexts, and non-English data scarcity (Yang et al. 2024). Beyond dataset limitations, measurement metrics lack comprehensiveness (Hu and Zhou 2024). For instance, hallucination assessment employs factual consistency metrics, while privacy evaluation requires analysis of information leakage. Recently proposed frameworks still rely on conventional metrics such as accuracy calculated on static datasets, failing to capture guardrail effectiveness and adaptiveness during diverse user interactions (Bassani and Sanchez 2024; Hu et al. 2024). Furthermore, evaluation on adversarial techniques also typically examines isolated attributes, overlooking how attacks may simultaneously compromise multiple guardrail objectives (Yi et al. 2024), further impeding development of standardized comprehensive assessment metrics.

4 Overcome and enhance guardrails

Implementing advanced safeguarding techniques, as discussed in Sect. 3, has played a crucial role in enhancing their security and reliability within LLMs. However, Shen et al. (2023) indicated that employing guardrails does not enhance the robustness of LLMs against attacks. They examined the external guardrails such as *ModerationEndpoint*, *OpenChatKit-Moderation Model*, and *Nemo*, showing that they only marginally reduce the average success rate of jailbreak attacks. Jailbreak attacks, referred to as “*jailbreaks*”, aim to exploit language models’ inherent biases or vulnerabilities by manipulating their responses. These successful attacks allow users to circumvent the model’s safeguard mechanisms, restrictions, and alignment, potentially leading to generating unconventional or harmful content or any content controlled by the adversary. By bypassing these constraints, jailbreaks empower the model to produce outputs that exceed the boundaries of its safety training and alignment.

Therefore, in this section, we explore current methods used to bypass the guardrails of LLM. In Table 3, we compare different jailbreaks on: (1) *Attacker access type*: white box, black box, and gray box. In a white-box scenario, the attacker has full visibility into the model’s parameters. A black-box situation restricts the attacker from observing the model’s

Table 3 Comparison among Different jailbreaks for (Guarded) LLMs

Attack	Access Type	Prompt Level	Core Technique	Stealthiness	GPT4 Evaluation	Targeted Property
GCG (Zou et al. 2023)	White	User	Greedy Gradient-based Search	Low	×	Harmful Content
PGD (Geisler et al. 2024)	White	User	Continuous Relaxation & Entropy projection	Low	×	Harmful Content
PRP* (Mangaokar et al. 2024)	White	User	In-context Learning & two-step prefix-based	Low	×	Harmful Content
AutoDAN-Liu* (Liu et al. 2023)	White	System+User	Hierarchical Genetic Algorithm	High	✓	Harmful Content
AutoDAN-Zhu* (Zhu et al. 2023)	White	User	Double-loop Optimization	High	✓	Harmful Content & prompt leaking
COLD-Attack* (Guo et al. 2024)	White	User	Language in dynamics	High	×	Harmful Content
ProMan (Zhang et al. 2023)	White	-	Generation Manipulation	-	×	Harmful Content & Privacy Leakage
JailBroken (Wei et al. 2024)	Black	System	Failure modes as guiding principles	Low	✓	Harmful Content & personally identifiable information leakage
DeepInception (Li et al. 2023)	Black	User	Nested instruction	Medium	✓	Harmful Content
DAN* (Shen et al. 2023)	Black	User	Characterizing in-the-wild prompt	High	✓	Harmful Content
ICA (Wei et al. 2023)	Black	User	In-context learning ability of LLM	Low	×	Harmful Content
SAP (Deng et al. 2023)	Black	User	In-context learning ability of LLM	Medium	×	Harmful Content
DRA (Liu et al. 2024)	Black	User	Making Them Ask and Answer	Low	✓	Harmful Content
CipherChat (Yuan et al. 2024)	Black	System	Long-tail: cipher	High	✓	Harmful Content
MultiLingual (Deng et al. 2023)	Black	User	Long-tail: low-resource	High	✓	Harmful Content
LRL (Yong et al. 2023)	Black	User	Long-tail: low-resource	High	✓	Harmful content
CodeChameleon (Lv et al. 2024)	Black	User	Long-tail: encrypts	High	✓	Harmful Content
ReNeLLM (Ding et al. 2023)	Black	User	Prompt rewriting & scenario nesting	High	✓	Harmful Content
PAIR (Chao et al. 2023)	Black	System	Automatic Iterative Refinement	High	✓	Harmful content
GPTFUZZER (Yu et al. 2023)	Black	User	Fuzzing	Low	✓	Harmful content
TAP (Mehrotra et al. 2023)	Black	System	Tree-of-thought reasoning	Medium	✓	Harmful content
Mosaic Prompts (Glukhov et al. 2023)	Black	User	Semantic censorship	High	×	Impermissible content
EasyJailbreak (Zhou et al. 2024)	Black	System+User	Unified framework for 12 jailbreaks	-	✓	Jailbreak attack evaluation

Table 3 (continued)

Attack	Access Type	Prompt Level	Core Technique	Stealthiness	GPT4 Evaluation	Targeted Property
PROMPTINJECT (Perez and Ribeiro 2022)	Black	User	Mask-based iterative strategy	Low	×	Goal hijacking & prompt leaking
IPI (Greshake et al. 2023)	Black	System	Indirect prompt injection	High	✓	Cyber threats like theft of data and denial of service etc
HOUYI (Liu et al. 2023)	Black	User	SQL injection & XSS attacks	Low	×	Prompt abuse & prompt leak
GA (Lapid et al. 2023)	Black	User	genetic algorithm	Low	×	Harmful content
GUARD (Jin et al. 2024)	Black	System+User	Role-playing LLMs	High	×	Harmful content
CIA (Jiang et al. 2023)	Black	User	Combination of multiple instructions	Medium	✓	Harmful content
Pelrine et al. (Pelrine et al. 2023) *	Grey	User	fine-tuning	Low	✓	Misinformation & Privacy Leakage
Zhang et al. (Zhan et al. 2023)	Grey	User	fine-tuning	Low	✓	Harmful content
Safety-tuned (Bianchi et al. 2023)	Grey	User	fine-tuning	Low	×	Harmful content
Janus inference (Chen et al. 2023)	Grey	System	Fine-tuning	Low	×	Privacy Leakage
Qi et al. (Qi et al. 2024)	Grey	User	fine-tuning	Low	×	Harmful content
Pelrine et al. (Pelrine et al. 2023)	Grey	User	poisoning knowledge retrieval	Medium	✓	Harmful content & fairness
PoisonedRAG (Zou et al. 2024)	Grey	System	poisoning knowledge retrieval	Low	✓	Hallucination
AutoPoison (Shu et al. 2024)	Grey	System	Content Injection	Low	×	Triggered response
LoFT (Shah et al. 2023)	Grey	System	fine-tuning	Low	✓	Harmful content
BadGPT (Shi et al. 2023)	Grey	System	Backdoor Attack	Medium	×	Harmful content
ICLAttack (Zhao et al. 2024)	Grey	System	Backdoor Attack	Low	×	Triggered response
ActivationAttack (Wang and Shu 2023)	Grey	System	Activation Steering	Low	×	Harmful & Biased content

* Claim to Jailbreak the Guardrails

outputs. In a grey-box context, the attacker has partial access, typically to some training data. (2) *Prompt level for manipulation*: user prompt or system prompt. User prompts are those where the input prompt is specified by the user, allowing for personalized or targeted inputs. On the other hand, system prompts are generated automatically by models and may include outputs that attackers craftily devise to deceive or manipulate the system's response. (3) *Core technique*: the main technique used to attack the LLM. (4) *Stealthiness*: high stealthiness represents that the attack is difficult to notice by a human, which is supposed to be some logical, semantic, and meaningful conversation rather than some gibberish. (5) *GPT4 Evaluation*: As many jailbreaks are not directly targeted for LLMs with guardrails, and GPT4 has its default guardrail, then evaluation on GPT4 can be seen as a surrogate metric for comparison. (6) *Target manipulated property of generated response*: toxicity, privacy, fairness, and hallucination

4.1 White-box jailbreaks

A white-box attack normally refers to a scenario in which the attacker has full access to the internal details of the model. Since even LLMs with guardrails can not fully protect against adversarial attacks, we introduce some techniques for attacking LLMs (with or without guardrails) under the white-box setting. Notably, most white-box attacks can be applied to the *black-box* scenario by employing their transferability on a white-box surrogate model.

4.1.1 Learning-based methods

The **Greedy Coordinate Gradient (GCG)** (Zou et al. 2023) method was designed to search a specific sequence of characters (an *adversarial suffix*). When the adversarial suffix is attached to different queries, it misleads the LLM to generate a response with harmful content. This approach integrates greedy search and gradient-based methods for discrete optimization to manipulate the model's outputs. It aims to optimize the likelihood that the model will generate an affirmative response, e.g., "Sure, this is...". To improve the computational efficiency of GCG, Geisler et al. (2024) revisited Projected Gradient Descent (**PGD**) for LLMs, which has been widely used for generating adversarial perturbations in other domains, by controlling the error introduced by the continuous relaxation for the input prompt, it can fool LLMs with the similar attack performance but with up to one order of magnitude faster. Previous techniques in traditional NLP, like Gradient-based Distributional Attack (**GBDA**) (Guo et al. 2021), can also be used to search adversarial suffixes. Specifically, it also applies continuous relaxation using Gumbel-Softmax (Jang et al. 2016), which allows for the manipulation of text inputs in a gradient-guided manner, maintaining the textual data's discrete nature while optimizing the adversarial objective. However, it fails to obtain high jailbreaking performance on the aligned LLMs.

On the other hand, the adversarial suffixes produced by GCG are mostly some garbled characters that are easily detectable by simple perplexity filter (Jain et al. 2023). **AutoDAN-Zhu** (Zhu et al. 2023) design a double-loop optimization method upon GCG to generate more stealthy jailbreak prompts. In addition, it also demonstrates the ability and interpretability to solve other new tasks like prompt leaking. Furthermore, **COLD-Attack** (Guo et al. 2024) adopts a new jailbreak, to automate the search of adversarial LLM attacks under a variety of restrictions such as fluency, stealthiness, sentiment, and left-right-coherence.

It performs efficient gradient-based sampling in the continuous logit space and relies on a guided decoding process to translate the continuous logit sequences back into discrete text.

Although the adversarial suffix can lead the base LLM to generate harmful responses, the LLM models with guardrails can easily detect it. Mangaokar et al. (2024) proposed a new attack strategy, named **PRP**, for attacking LLM models with guardrails mainly. Specifically, it leverages a two-step prefix-based attack, including universal adversarial prefix construction and prefix propagation to the response. Inserting the universal prefix into the response can elicit the guardrail for outputting the harmful content. After the universal prefix generation, the corresponding propagation prefix can be created through a few in-context templates. Such in-context learning enables the LLM to initially output the pre-computed or desired adversarial prefix, eventually making PRP jailbreak the LLM models with guardrails. Subsequently, **AutoDAN-Liu** (Liu et al. 2023) proposed to generate stealthy jailbreak prompts automatically, it utilizes a hierarchical genetic algorithm to bypass the ethical guidelines and safety measures of LLMs. This method is grounded in optimization techniques inspired by natural selection. It iteratively refines generations of prompts to circumvent built-in safeguards effectively. Through this evolutionary process, AutoDAN-Liu generates stealthy prompts that subtly avoid triggering the model's protective mechanisms.

4.1.2 LLM generation manipulation

On the other hand, except for jailbreaking via some learning-based method, ProMan (Zhang et al. 2023) was proposed to directly manipulate the generation process of those open-source LLMs and enforce the LLMs to generate specific tokens at specific positions, therefore cheating the LLMs to generate undesired response, including harmful or sensitive information or even private data.

Although the current literature only describes limited white-box jailbreaks, it is still possible to bypass the guardrails if full access is provided. For example, suppose the attacker knows the guardrail used for the targeted LLM is Llama Guard, and the adversary has full access to the fine-tuned Guard model. In that case, the previous white-box attacks using optimization can be further extended: the optimization space will be further narrowed by adding an extra constraint, i.e., the adversarial input and/or the resulting response of LLMs are supposed to be evaded by the guardrail. In other words, a successful jailbreak should satisfy the safety conditions of the targeted LLM and the Guard models.

4.2 Black-box jailbreaks

Unlike white-box attacks, which necessitate access to model weights and tokenizers, black-box attacks operate under the premise that adversaries lack knowledge of the LLM's internal architecture or parameters. Therefore, they are more common. In this subsection, jailbreak attacks conducted within a black-box setting are classified into four categories: i) *manually designed jailbreaks*, ii) *attacks exploiting long-tail distribution*, iii) *optimization-based methods for generating jailbreaks*, and iv) *unified framework for jailbreaking*.

4.2.1 Delicately designed jailbreaks

The phenomenon of jailbreak attacks against state-of-the-art large LLMs was investigated in **JailBroken** (Wei et al. 2024), explicitly focusing on models such as GPT-4, GPT-3.5 Turbo, and Claude v1.3. This work identifies two primary reasons for the successful attack: competing training objectives and instruction tuning objectives. The authors propose these two failure modes as guiding principles for designing new jailbreak attacks. Using carefully engineered objectionable prompts, they empirically evaluate these attacks against the aforementioned safety-trained LLM models. The results indicate a high success rate regarding a large number of the attacks.

In this line, due to the constant evolution of ethical and legal constraints embedded within LLM safeguards, jailbreak attempts employing direct instructions (Wei et al. 2024; Chao et al. 2023) are typically easily identified and rejected. It is motivated by the Milgram shock experiment (Milgram 1963, 1975) and its adaptation to LLMs, which follow authoritative commands to produce harmful content. A prompt-based jailbreak method, called **DeepInception** (Li et al. 2023), was devised for conducting a black-box attack on LLMs. This involves injecting an inception mechanism into a LLM and effectively hypnotizing it to act as a jailbreaker. DeepInception explicitly constructs a nested scene to serve as the inception for guiding the behavior of the LLM. This nested scene facilitates an adaptive approach to circumvent safety constraints in a normal scenario, opening up the potential for subsequent jailbreaks. Specifically, DeepInception utilizes the personification capability of LLMs, along with their tendency to follow instructions, to generate diverse scenes or characters. Shen et al. (2023) furthered this by creating prompts that encourage ChatGPT to act as a **DAN** ("Do Anything now"). As implied by their designation, LLMs are now capable of boundless functions. They are no longer bound by the customary rules that govern AI systems.

While existing attack methods are typically applied to new conversations devoid of context, the potential of In-Context Learning (ICL) was delved into the influence of the alignment ability of LLMs. Leveraging these insights, the study introduces the **In-Context Attack (ICA)** (Wei et al. 2023). ICA is tailored to construct malicious contexts to direct models to produce harmful outputs. The efficacy of in-context demonstrations in aligning LLMs is demonstrated, and implementing these methods is straightforward. Additionally, Deng et al. (2023) proposed a semi-automatic attack framework named Semi-Automatic Attack Prompt (**SAP**), it combines manual and automatic methods to generate prompts to mislead LLMs to output harmful content. Specifically, they manually construct high-quality prompts as an initial prompt set and then iteratively update them through in-context learning with LLMs. Through this red-teaming attack, extensive high-quality attack prompts can be efficiently generated. Liu et al. (2024) proposed a novel universal jailbreak approach named **DRA** (Disguise and Reconstruction Attack. This method involves concealing harmful instructions via disguise, prompting the model to uncover and reconstruct the original harmful instruction within its generated output, thus navigating around traditional security measures. In this way, the harmful input can be disguised from the input filter, which then guides the target to reconstruct the attack to obtain the desired response from the adversary.

4.2.2 Exploiting long-tail distribution

Jailbreaks relying on long-tail distributed encoding convert the original query into rare or unique data formats such as ciphers (Yuan et al. 2024), low-resource languages (Deng et al. 2023), and personalized encryption methods (Lv et al. 2024). The safety vulnerability of LLMs when user queries are encrypted was investigated in **CipherChat** (Yuan et al. 2024). The framework involves encoding malicious unsafe text using LLMs and assessing the safety of the decoded responses. CipherChat is designed with three key elements in its system prompt to ensure effective communication through ciphers: (1) behavior assignment, (2) cipher teaching, and (3) enciphered unsafe demonstrations. It enables users to interact with LLMs using cipher prompts, system role descriptions, and few-shot enciphered demonstrations. Furthermore, the authors introduce SelfCipher, which utilizes a hidden cipher embedded within LLMs to circumvent safety features more efficiently than existing ciphers.

Afterward, despite the widespread use of English globally, there is growing concern that the safety of LLMs is predominantly assessed in English alone. However, **MultiLingual** (Deng et al. 2023) takes a significant stride forward by investigating the safety levels of LLMs across various languages, including those with limited linguistic resources. This research delves into the vulnerabilities of LLMs from two perspectives: unintentional and intentional scenarios. In the unintentional scenario, queries translated into non-English languages unexpectedly expose users to unsafe content. Conversely, the intentional scenario involves using translated multilingual “jailbreak” prompts. Similarly, Low Resource Languages-Combined Attacks (Yong et al. 2023) (**LRL**) also underlines the cross-lingual vulnerability of GPT-4. By translating unsafe English prompts into less commonly used languages, they successfully circumvent protective measures to elicit harmful responses. Kang et al. (2023) show that instruction-following language models using the TEXT-DAVINCI-003 prompt could potentially be employed to produce malicious content.

A hypothesis regarding LLMs’ safety mechanisms was proposed in subsequent research, suggesting that LLMs first detect intent before generating responses. Building on this hypothesis, a framework was introduced known as **CodeChameleon**, which encrypts and decrypts queries into a format challenging for LLMs to detect Lv et al. (2024). Four distinct encryption functions are employed during the encryption stage based on reverse order, word length, odd and even positions, and binary tree structure. Subsequently, the decryption functions are incorporated into the instructions as code blocks. During inference, these decryption functions assist LLMs in understanding the encrypted content. Extensive testing demonstrates that CodeChameleon effectively circumvents LLMs’ intent recognition.

4.2.3 Optimization-based approaches

In contrast to conventional adversarial examples, such jailbreaks are usually created through human ingenuity, strategically devising situations that naturally mislead the models (Wei et al. 2024), rather than relying on automated techniques. Consequently, crafting them demands considerable manual labor. The adversarial prompts generated **Greedy Coordinate Gradient (GCG)** (Zou et al. 2023) exhibit a high degree of universality and transferability, particularly to other fully black-box models.

To avoid limitations regarding intricate manual design (Wei et al. 2024; Yuan et al. 2024) and require optimization on other white-box models, compromising generalization or effi-

ciency (Zou et al. 2023), a method known as **ReNeLLM** was introduced (Ding et al. 2023). ReNeLLM is an automatic jailbreak prompt generation framework, which generalizes jailbreak prompt attacks into two aspects: (1) Prompt Rewriting and (2) Scenario Nesting.

Following this, **Prompt Automatic Iterative Refinement (PAIR)** proposed an automated red teaming method for jailbreaking LLMs (Chao et al. 2023), which represents a significant improvement of over ten thousand times compared to existing attacks, such as jailbreaks identified through Greedy Coordinate Gradient (GCG) (Zou et al. 2023). The authors aim to find a balance between prompt-level attacks (Dinan et al. 2019), which are labor-intensive but scalable, and token-level attacks (Maus et al. 2023), which are uninterpretable and inefficient in terms of queries. PAIR devised a protocol leveraging a language model to craft prompt-level attacks that are both semantic and human-interpretable. This involves an automated system where the attacker language model learns from prior prompts and responses to refine based on a judge score and generate new prompts. Through in-context learning, PAIR enabled the language model to enhance the quality of generated candidate queries autonomously.

Drawing inspiration from AFL fuzzing, **GPTFUZZER** was introduced, a black-box jailbreak fuzzing framework to autonomously generate jailbreak prompts (Yu et al. 2023). GPTFUZZER aims to combine the efficacy of human-written prompts with the scalability and flexibility of automated systems to bolster the assessment of vulnerabilities in LLMs. The framework is built upon a seed selection strategy, mutate operators, and a judgment model. By harnessing these elements, GPTFUZZER can systematically detect and exploit vulnerabilities in LLMs.

Building upon prior automated methodologies, **Tree of Attacks with Pruning (TAP)** introduced a novel approach for generating jailbreaks (Mehrotra et al. 2023). TAP leverages an LLM to iteratively refine candidate prompts using tree-of-thought reasoning until a successful jailbreaking prompt is generated. The framework involves three key components: an attacker LLM tasked with generating jailbreaking prompts using tree-of-thought reasoning, an evaluator responsible for assessing the generated prompts and determining the success of the jailbreaking attempt, and a target LLM that serves as the subject of the jailbreaking endeavor. Lapid et al. (2023) employed the genetic algorithm (GA) for generating a universal adversarial suffix under the black-box setting. Instead of maximizing the targeted token likelihood in GCG, they proposed using random subset sampling for fitness approximation by minimizing the cosine similarity between benign input embedding and adversarial input embedding. Experiments illustrate high transferability across different LLMs. Furthermore, Jin et al. (2024) proposed a role-playing system named Guideline Upholding through Adaptive Role-play Diagnostics (**GUARD**), which allocates four distinct roles to user LLMs to collaborate on new jailbreaks. By collecting some existing jailbreak prompts into a knowledge graph and using Chain-of-Thought to align with the specific functions and objectives for each role, they can generate a higher jailbreak success rate and a lower perplexity score than GCG (Zou et al. 2023) and AutoDAN (Zhu et al. 2023).

4.2.4 Unified framework for jailbreaking

A recent development, EasyJailbreak (Zhou et al. 2024), presents a comprehensive framework to evaluate jailbreak attacks on LLMs. This framework integrates four pivotal components: Selector, Mutator, Constraint, and Evaluator. This approach allows researchers to

concentrate on crafting unique components, thus minimizing the effort required for development. Moreover, it demonstrates broad model compatibility, accommodating various models, including open-source alternatives like LLaMA2 and proprietary ones like GPT-4.

4.2.5 Prompt injection for desired responses

Prompt Injection in LLMs involves the malicious alteration of input provided to the model, commonly achieved by substituting original instructions with carefully crafted user input (Shayegani et al. 2023). This manipulation occurs within the framework of supplying prompts to the LLM, guiding its responses or behaviors. Prompt injection attacks present a significant cybersecurity risk as they can result in creating unauthorized content, circumventing content moderation protocols, exposing sensitive data, or even facilitating the dissemination of malicious code or malware. This vulnerability is particularly prominent in LLMs that employ prompt-based learning approaches, rendering them susceptible to exploitation by malicious attackers. Given the significant role of prompt in shaping LLM output, prompt injection manipulation can have widespread implications for attacking LLMs.

Since most LLMs, such as ChatGPT, are closed-source platforms, much of the research centers on utilizing prompt engineering techniques to induce ChatGPT to generate inappropriate content. The framework known as **PROMPTINJECT** (Perez and Ribeiro 2022) was proposed as a straightforward alignment mechanism for generating iterative adversarial prompts through masks. This approach involves assembling prompts to facilitate a quantitative assessment of the robustness of LLMs against adversarial prompt attacks. The study focuses primarily on evaluating the susceptibility of GPT3 to such attacks, accomplished through simplistic handcrafted inputs. The analysis concentrates on two types of attacks: goal hijacking and prompt leaking. Goal hijacking involves introducing a malicious string, termed a rogue string, designed to divert the model into generating a particular sequence of characters. Conversely, prompt leaking pertains to the possibility of revealing a private value embedded within a confidential prompt, which should not be disclosed externally under any circumstances. Following this, the concept of **Indirect Prompt Injection (IPI)** (Greshake et al. 2023) was introduced, referring to an uninvestigated attack vector where retrieved prompts can function as “arbitrary code”, thus compromising LLM-integrated applications. The authors demonstrate these attacks against real-world systems like Bing Chat, code-completion engines, and GPT-4.

Inspired by traditional injection attacks, a novel black-box prompt injection attack technique called **HOUYI** (Liu et al. 2023) was introduced. HOUYI comprises three essential components: a preconstructed prompt, an injection prompt, and a malicious question, each tailored to achieve the adversary’s goals. Two significant exploit scenarios were identified: prompt abuse and prompt leak. The application of HOUYI to a sample of 36 real-world LLM-integrated applications revealed that 31 of these applications are vulnerable to prompt injection. Glukhov et al. (2023) raised a concept of semantic censorship, which falls into the category of guardrail using a universal algorithm to determine whether the content generated by an LLM is permissible based on semantic content alone. Accordingly, they proposed a novel attack named **Mosaic Prompts**; it leverages the ability of a user to query an LLM multiple times in independent contexts to construct impermissible outputs from a set of permissible ones. This indicates a significant limitation of output censorship, as it cannot provide safety or security guarantees without imposing severe restrictions on model useful-

ness. Moreover, Compositional Instruction Attack (CIA) (Jiang et al. 2023) capitalizes on LLMs' failure to detect underlying harmful intents when instructions are composed of multiple elements, thus revealing significant vulnerabilities in LLM security mechanisms. They outline two specific strategies, Talking-CIA (T-CIA) and Writing-CIA (W-CIA), developed to automate the generation of these deceptive instructions. T-CIA leverages psychological principles to align the model's response persona with the harmful intent, bypassing LLMs' ethical constraints. Conversely, W-CIA disguises harmful prompts as creative writing tasks, exploiting LLMs' lack of judgment on fictional content to elicit dangerous outputs.

4.3 Gray-box jailbreaks

In Pelrine et al. (2023), the authors highlight that beyond the white-box approach, which involves full access to a model's parameters, and the more limited black-box method, there's also 'grey-box' access. This middle ground could be crucial in uncovering additional vulnerabilities in the safeguard systems of LLMs. This section will present studies on 'grey-box' attack methods, encompassing strategies like fine-tuning, retrieval-augmented generation, and backdoor attacks.

4.3.1 Fine-tuning attacks

Fine-tuning technology enables users to customize pre-trained LLMs effectively. However, when these fine-tuning privileges are extended to end-users, the existing guardrails may not be sufficient to prevent harmful behaviors. The attacks by fine-tuning the LLMs can also be called 'grey-box' attacks. Zhan et al. (2023) suggested that fine-tuning could mitigate Reinforcement Learning with Human Feedback (RLHF) safeguards, commonly employed in LLMs to minimize harmful outputs. Their research revealed that even ChatGPT 4 could have its protections removed by fine-tuning. Through experiments, they demonstrated a success rate of 95% in generating harmful responses from ChatGPT 4, using just 340 examples for fine-tuning. The experiments from Pelrine et al. (2023) also indicate that fine-tuning a model with only 15 harmful or 100 benign examples can compromise the safeguards of GPT-4, leading to a variety of harmful outputs. Bianchi et al. (2023) also examines the potential safety risks associated with excessive instruction tuning in LLM, illustrating that models excessively tailored to specific instructions may still generate harmful content. To counteract these risks, the researchers suggest developing a safety-focused tuning dataset to balance the dual objectives of maintaining model performance while enhancing safety measures. Furthermore, the research by Chen et al. (2023) highlights the risks of fine-tuning language models using small datasets containing personally identifiable information (PII). Initially, it focuses on a simple approach where a language model is fine-tuned with a small dataset rich in text-based PII, which results in the model being more likely to divulge PII upon prompting. Then, the researchers introduced the "Janus" methodology, which centers around defining a PII recovery task followed by few-shot fine-tuning. Experimental findings demonstrate that fine-tuning GPT-3.5 with just 10 PII examples markedly increases the model's ability to expose PII. Qi et al. (2024) found that additional training of the model can compromise the effectiveness of established guardrails. They bypass the GPT-3.5 Turbo's safety guardrails by fine-tuning it with just ten specific examples and successfully make the model entirely susceptible to harmful instructions.

4.3.2 Retrieval-augmented generation (RAG)

RAG for LLMs aims to improve the response of LLMs by incorporating external datasets during inference. It integrates context and up-to-date or relevant information in the prompt to enhance the LLM's performance. Pelrine et al. (2023) finds that employing the prompt injection techniques suggested by Perez and Ribeiro (2022) indicated that polluting the external dataset by injecting a malicious instruction could successfully invalidate ChatGPT 4's safety protection. They also demonstrated that if biased system messages accompany the upload of factual data, it can bias the responses of ChatGPT. Zou et al. (2024) also proposed to inject toxic texts into the knowledge database to compromise LLMs. They developed these poisoned texts by forming them to solve an optimization problem aimed at generating a target response chosen by the attacker. Their experiments showed that by injecting just five tainted texts tailored to a specific question, they were able to attain a 90% attack success rate.

4.3.3 Backdoor attack

The backdoor attack on the neural language process task is to manipulate the model to produce specific outputs when triggered (Cai et al. 2022). It typically occurs during the pre-training and adaptation tuning, where the backdoor trigger gets embedded (Chen et al. 2021). These manipulations should maintain the model's performance and evade detection by human inspection. The backdoor is triggered exclusively when input prompts to LLMs include the embedded trigger, causing the compromised LLMs to act maliciously as intended by the attacker. Shu et al. (2024) propose Auto Poinson to incorporate training examples that reference the desired target content into the system, triggering similar behaviors in downstream models. Shah et al. (2023) introduces LoFT (Local Proxy Fine-tuning) to fine-tuning smaller, local proxy models to develop attacks that are more likely to transfer successfully to larger, more complex LLMs. This technique leverages the target LLMs to produce prompts closely aligned with harmful queries, effectively gathering prompts from a localized vicinity around these queries. A set of parameters in the proxy LLM is then fine-tuned, guided by the responses of the target LLM to these analogous prompts. Ultimately, this fine-tuned proxy model is deployed to attack the target LLMs. The study demonstrates that this method improves the transferability of attacks. Shi et al. (2023) proposed the BadGPT, a backdoor attack targeting RL fine-tuning in language models. It injects a backdoor trigger into the reward model during the fine-tuning stage, allowing for compromising the fine-tuned language model. Zhao et al. (2024) then proposed ICLAttack, which fine-tunes models by targeting in-context learning for backdoor attacks. This method focuses on two prompt-level strategies: introducing compromised examples within the prompt's demonstration set and modifying the prompts. This technique operates at the prompt level, eliminating the necessity to train new LLMs altogether. On the other hand, Wang and Shu (2023) pointed out that poisoning the training dataset or introducing harmful prompts affects the adaptability of the attacks, rendering them more prominent and more accessible to identify. They propose using activation steering without optimization to target four key aspects of LLMs: truthfulness, toxicity, bias, and harmfulness.

4.4 Techniques for strengthening LLMs

This section discusses techniques that may help construct more powerful defenses for guardrails or more robust LLMs.

4.4.1 Detection-based methods: guardrail enhancement

To detect the harmful information in the user's input, **PPL** (Alon and Kamfonas 2023) calculates the perplexity of a provided input to decide whether a user's request should be accepted or rejected. **SmoothLLM** (Robey et al. 2023) borrowed the idea of randomized smoothing literature (Cohen et al. 2019), it randomly alters multiple versions of a given input and then combines the respective predictions to identify adversarial inputs. Some researchers have explored how In-Context Learning (ICL) can impact the alignment capabilities of LLMs. **In-Context Defense (ICD)** (Wei et al. 2023) method is designed to bolster model resilience by demonstrations of rejecting to answer harmful prompts via in-context demonstration.

To defend LLM attacks, **LLM SELF DEFENSE** (Helbling et al. 2023) was proposed first. Specifically, by incorporating the generated content into a pre-defined prompt and using another instance, LLM, to analyze the text, it constructs an extra guardrail filter for preventing harmful content. Furthermore, Cao et al. (2023) proposed Robustly Aligned LLM (**RA-LLM**) to defend against potential alignment-breaking attacks. Unlike the previous alignment check, which uses the alignment check function to decide whether to reject the response, the proposed Robust Alignment Check Function adds several extra random droppings on the request. It usually checks whether the corresponding response can still pass the alignment check function AC. Then, Chen et al. (2023) designed a moving target defense (**MTD**) to enhance the LLM system. Compared to previous guardrail methods that decide whether the input/output is safe, MTD calculates a composite score for each response by combining its *quality* and *toxicity* metrics. It employs randomization to select a response that qualifies both response metrics, eventually providing a solid moving target defense for the LLMs.

4.4.2 Mitigation-based methods: affirmative response generation

As shown in. Jain et al. (2023), besides perplexity filtering, input preprocessing like **Retokenization** and **Paraphrase** can also successfully compromise the effectiveness of some attacks like GCG (Zou et al. 2023). However, adversarial training, though once favored for safeguarding image classifiers, faces diminished appeal for LLMs due to the prohibitive expenses associated with both model pre-training and the creation of adversarial attacks, rendering large-scale adversarial training impractical. Finding a good approximation for robust optimization objectives that allow for successful adversarial training remains an open challenge. Further, Li et al. (2023) introduced a novel inference method, Rewindable Auto-regressive Inference (**RAIN**) enables pre-trained LLMs to assess their own outputs and leverage the assessment outcomes to inform and steer the backtracking and generating content to enhance AI safety. Contrary to Reinforcement Learning from Human Feedback (RLHF), RAIN dispenses with the requirement for extra model upkeep and bypasses the accumulation of gradient data and computational graphs. Still, it must pay the extra but

acceptable cost of the auto-regressive inference. Additionally, Zhang et al. (2023) proposed to integrate goal prioritization (**GP**) at both training and inference stages. It analyzes the reason behind successful jailbreaking: the conflict between two goals: helpfulness (providing helpful responses to user queries) and safety (providing harmless and safe responses to user queries). The jailbreak attack success rate can be notably decreased by plugging in the goal prioritization for these two properties into the inference alone or with training.

Further, **Self-Reminder** (Xie et al. 2023) suggests that adding self-reminder prompts can be an effective defense. They speculate that initiating ChatGPT with a ‘system mode’ prompt at the most external level to remind it of its role as a responsible AI tool could reduce its vulnerability to being malevolently steered by user inputs at a deeper level. Therefore, by concatenating an extra system prompt after the user’s query that reminds the LLMs to respond responsibly, the experimental results showed that self-reminders significantly reduce the success rate of jailbreak attacks. Then, Ge et al. (2023) proposed a multi-round automatic red-teaming framework **MART** to improve the scalability of safety alignment. Two players, i.e., an adversarial LLM and a target LLM, iteratively interplay with each other. The adversarial LLM aims to generate challenging prompts that provoke unsafe responses from the target LLM. Concurrently, the target LLM is refined with data that aligns with safety standards based on these adversarial prompts. Through several rounds of red-teaming, the enhanced target LLM continues to bolster its defenses through safety-specific fine-tuning. Further, Zhou et al. (2024) proposed the first adversarial objective aimed at protecting language models from jailbreaking attacks, along with a novel algorithm, Robust Prompt Optimization (**RPO**). This strategy employs gradient-based token optimization (similar to GCG) to ensure the generation of harmless outputs. RPO represents the initial approach in jailbreaking defense (like adversarial training in vision) that enhances robustness comprehensively and effectively and at only a minor cost to normal use. **SafeDecoding** (Xu et al. 2024) found that despite the likelihood of tokens signifying harmful content being higher than those for harmless responses, safety disclaimers continue to emerge among the highest-ranking tokens when sorted by probability in descending order. Thus, in the training phase, the model will be fine-tuned with a few safety measures, and then, during the inference, SafeDecoding further constructs the new token distribution. The crafted probability distribution reduces the chances of tokens that resonate with the attacker’s objectives and enhances the probabilities of tokens that align with human values.

5 Discussions: a complete guardrail

Based on the discussions about tackling individual requirements in Sects. 3 and 4, this section advocates building a guardrail by systematically considering multiple requirements. We discuss four topics: conflicting requirements (Sect. 5.1), multidisciplinary approach (Sect. 5.2), implementation strategy (Sect. 5.3), rigorous engineering process (Sect. 5.4), and safeguards for LLM Agents (Sect. 5.5) and emerging challenges of MLLM guardrails (Sect. 5.6).

5.1 Conflicting requirements

This section discusses the tension between safety and intelligence as an example of the conflicting requirements. Conflicting requirements are typical, including, e.g., fairness and privacy (Xiang 2022), privacy and robustness (Song et al. 2019), and robustness and fairness (Bassi et al. 2024). Integrating guardrails with LLMs may lead to a discernible conservative shift in the generation of responses to open-ended text-generation questions (Röttger et al. 2023). The shift has been witnessed in ChatGPT over time. Chen et al. (2023) documented a notable change in ChatGPT's performance between March and June 2023. Specifically, when responding to sensitive queries, the model's character count decreased significantly, plummeting from an excess of 600 characters to approximately 140. Additionally, in the context of opinion-based questions and answers surveys, the model is more inclined to abstain from responding.

Given the brevity and conservativeness of responses generated by ChatGPT, the following question arises: How can exploratory depth be maintained in responses, particularly for open-ended text generation tasks? Furthermore, does the application of guardrails constrain ChatGPT's capacity to deliver more intuitive responses? On the other hand, Narayanan and Kapoor (2023) critically examined this paper and emphasized the difference between an LLM's capabilities and its behavior. Although capabilities typically remain constant, behavior can alter due to fine-tuning, which can be interpreted as the "uncertainty" challenges in LLMs. They suggest that GPT-4's performance changes are likely linked more to evaluation data and fine-tuning methods rather than a decline in its fundamental abilities. They also acknowledge that such behavioral drift challenges the development of reliable chatbot products. The adoption of guardrails has also led to the model adopting a more concise communication approach, offering fewer details and electing non-response in specific queries. The decision "to do or not to do" can be challenging when designing the guardrail. While the most straightforward approach is to decline an answer to any sensitive questions, is it the most intelligent one? That is, *we need to determine if the application of guardrail always has a positive impact on LLMs that is within our expectation.*

5.1.1 Our perspective

Prior research suggested incorporating a creativity assessment mechanism into the guardrail development for LLMs. To measure the creativity capability of LLMs, Chakrabarty et al. (2023) employed the Consensual Assessment Technique (Amabile 1982), a well-regarded approach in creativity evaluation, focusing on several key aspects: fluency, flexibility, originality, and elaboration, which collectively contribute to a comprehensive understanding of the LLMs' creative output in storytelling. Narayanan and Kapoor (2023) showed that although some LLMs may demonstrate adeptness in specific aspects of creativity, there is a significant gap between their capabilities and human expertise when evaluated comprehensively. Moreover, for addressing trade-offs between safety and utility, we advocate exploring the theoretical boundaries between these properties (Huang et al. 2023). We propose developing formal frameworks to characterize trade-off spaces, using statistical methods to determine whether certain combinations of guardrail properties are fundamentally unattainable. A potential direction would be developing multi-objective optimization approaches that explicitly quantify both safety and utility aspects to consider multiple undesirable prop-

erties simultaneously rather than treating them as separate concerns (Rame et al. 2023; Wachi et al. 2024). This would enable constructing Pareto-optimal solution sets that visualize concrete trade-offs and allow developers to make informed choices based on application requirements, moving guardrail design from intuitive engineering to rigorous optimization grounded in theoretical understanding.

5.2 Multidisciplinary approach

While current LLM guardrails include mechanisms to detect harmful content, they still risk generating biased or misleading responses. It is reasonable to expect future guardrails to integrate harm detections and other mechanisms to deal with, e.g., ethics, fairness, and creativity. In the introduction, we have provided three categories of requirements to be considered for a guardrail. Moreover, LLMs may not be universally effective across all domains, and it has been a trend to consider domain-specific LLMs (Pal et al. 2023). In domain-specific scenarios, specialized rules may conflict with the general principles. For instance, in crime prevention, the use of certain terminologies that are generally perceived as harmful, such as ‘guns’ or ‘crime,’ is predominant and should not be precluded. To this end, the concrete requirements for guardrails will differ across different LLMs, and research is needed to *scientifically* determine requirements. The above challenges (multiple categories, domain-specific, and potentially conflicting requirements) are compounded by the fact that many requirements, such as fairness and toxicity, are hard to define precisely, especially without a concrete context. The existing methods, such as the popular one that sets a threshold on predictive toxicity level (Perez et al. 2022), do not have *valid justification and assurance*.

5.2.1 Our perspective

Developing LLMs ethically involves adhering to fairness, accountability, and transparency. These principles ensure that LLMs do not perpetuate biases or cause unintended harm. The works by e.g., Sun et al. (2023) and Ovalle et al. (2023) provide insights into how these principles can be operationalized in the context of LLMs. Establishing community standards is vital for the responsible development of LLMs. These standards, derived from a consensus among stakeholders, including developers, users, and those impacted by AI, can guide LLMs’ ethical development and deployment. They ensure that LLMs are aligned with societal values and ethical norms, as discussed in broader AI ethics literature (ActiveFence 2023). Moreover, the ethical development of LLMs is not a one-time effort but requires ongoing evaluation and refinement. These tasks involve regular assessment of LLMs outputs, updating models to reflect changing societal norms, and incorporating feedback from diverse user groups to ensure that LLMs remain fair and unbiased.

Socio-technical theory (Trist and Bamforth 1957), in which both ‘social’ and ‘technical’ aspects are brought together and treated as interdependent parts of a complex system, have been promoted (Filgueiras et al. 2023; Jr et al. 2020) for machine learning to deal with properties related to human and societal values, including e.g., fairness (Dolata et al. 2022), biases (Schwartz et al. 2022), and ethics (Mbiazi et al. 2023). To manage the complexity, the whole system approach (Crabtree et al. 2011), which promotes an ongoing and dynamic way of working and enables local stakeholders to come together for an integrated solution, has been successfully working on healthcare systems (Brand et al. 2017). We believe a

multi-disciplinary group of experts will work out and rightly justify and validate the concrete requirements for a specific context by applying the socio-technical theory and the whole system approach.

5.3 Neural-symbolic approach for implementation

Existing guardrail frameworks such as those introduced in Sect. 3 employ a language (such as RAIL or Colang) to describe the behavior of a guardrail. A set of rules and guidelines are expressed with the language, so each is applied independently. It is unclear if and how such a mechanism can deal with more complex cases where the rules and guidelines conflict. As mentioned in Sect. 5.2, such complex cases are common in building guardrails. Moreover, it is unclear if they are sufficiently flexible and capable of adapting to semantic shifts over time and across different scenarios and datasets.

5.3.1 Our perspective

First, a principled approach is needed to resolve conflicts in requirements, as suggested in van Lamsweerde et al. (1998) for requirement engineering, which is based on the combination of logic and decision theory. *Second*, a guardrail requires the cooperation of symbolic and learning-based methods. For example, we may expect that the learning agents deal with the frequently-seen cases (where there are plenty of data) to improve the overall performance w.r.t. the requirements mentioned above, and the symbolic agents take care of the rare cases (where there are few or no data) to improve the performance in dealing with corner cases in an interpretable way. Due to the complex conflict resolution methods, more closely coupled neural-symbolic methods might be needed to deal with the tension between effective learning and sound reasoning, such as those Type-6 systems (Lamb et al. 2021) that can deal with true symbolic reasoning inside a neural engine, e.g., Pointer Networks (Vinyals et al. 2015).

5.4 Systems development life cycle (SDLC)

The criticality of guardrails requires a careful engineering process. For this, a revisit of the SDLC, which is a complex project management model to encompass guardrail creation from its initial idea to its finalized deployment and maintenance, has the potential, and the V-model (Oppermann 2023), which builds the relations of each development process with its testing activities, can be helpful to ensure the quality of the final product.

5.4.1 Our perspective

Rigorous verification and testing will be needed (Huang et al. 2023), which requires a comprehensive set of evaluation methods. Certification with statistical guarantees can be helpful for individual requirements, such as the randomized smoothing (Cohen et al. 2019). For the evaluation of multiple, conflicting requirements, a combination of the Pareto front-based evaluation methods for multiple requirements (Ngatchou et al. 2005) and the statistical certification for a single requirement is needed. The Pareto front is a concept from the field of multi-objective optimization. It represents a set of non-dominated solutions, where

no other solutions in the solution space are better when all objectives are considered. Statistical certification involves using statistical methods to ensure that a single requirement meets a specified standard with a certain confidence level. It is typically applied when there is uncertainty in the measurements, or the requirement is subject to variability. Combining these techniques can find the trade-offs, provide confidence in the viability of solutions concerning individual requirements, and support more informed and adaptive decision-making processes. Finally, attention should also be paid to understanding the theoretical limits of the evaluation methods, e.g., randomized smoothing causes a fairness problem (Mohapatra et al. [bbbb](#)).

While these conflicts may not be entirely resolvable, particularly within a general framework applicable across various contexts, more targeted approaches in specific scenarios might offer better conflict resolution. Such approaches demand ongoing research to develop concrete principles, methods, and standards that a multidisciplinary team can implement and adhere to. While effective in particular situations, Guardrails are not a universal solution that addresses all potential conflicts. Instead, they should be designed to manage specific, well-defined scenarios.

5.5 Safeguards for LLM agents

In the rapidly evolving field of LLM, more autonomous entities extend the capabilities of LLMs by integrating decision-making and action-initiating capacities. *LLM agents* process and generate language and use this capability to perform actions in the digital or physical world. These LLM agents typically encompass five fundamental modules: *LLMs, planning, action, external tools, and memory and knowledge*. Tang et al. ([2024](#)). While LLMs respond passively to user queries, LLM agents can take proactive steps based on their understanding or directives. This increased autonomy raises concerns about unintended consequences, especially in sensitive domains like scientific research.

5.5.1 Our perspective

Due to their autonomy, LLM agents introduce higher complexity and unpredictability. The integration of decision-making processes means they might initiate actions that are hard to foresee or control, potentially leading to ethical and practical risks. Different from safeguard LLMs, the safety of agents interacting with various tools and environments is often overlooked, leading to potential harmful outputs, as highlighted in studies such as ToolEmu (Ruan et al. [2023](#)), AgentMonitor (Naihin et al. [2023](#)), and R-Judge (Yuan et al. [2024](#)). For LLM agents, "safeguard" means implementing stricter controls and oversight to manage their broader capabilities effectively.

5.6 Challenges of MLLM guardrails

As multimodal large language models (MLLMs) become increasingly prominent in real-world applications, ranging from AI assistants with visual perception to audio-driven instruction following, ensuring their safety, reliability, and compliance with ethical policies has emerged as a crucial research frontier. Compared to their unimodal (text-only) counterparts, MLLMs present unique challenges in guardrail design due to their inherently complex

input modalities, multimodal reasoning capabilities, and vulnerability to cross-modal adversarial manipulation (Yin et al. 2025).

Recent advances have demonstrated specialized guardrail frameworks across different modalities. In the vision-language domain, LlavaGuard (Helff et al. 2024) introduced a safety evaluation system built upon LLaVA, capable of policy-aware classification of image content into nine risk categories with structured JSON responses and natural language explanations. SafeWatch (Chen et al. cccc) extended guardrail capabilities to video content, where long-form temporal inputs pose additional challenges, by implementing Parallel Equivalent Policy Encoding for disentangled policy representation and Policy-Aware Adaptive Pruning for eliminating irrelevant video tokens, while contributing the SafeWatch-Bench dataset for video-based safety auditing. Simultaneously, in the audio-language space, SpeechGuard (Peri et al. 2024) addressed the adversarial robustness of speech-language models by proposing time-domain noise flooding as a lightweight defense mechanism against white-box and black-box adversarial audio perturbations that could bypass safety alignment and elicit harmful responses.

5.6.1 Our perspective

Despite these advances, the current ecosystem of MLLM guardrails remains fragmented and lacks general-purpose, modality-agnostic frameworks, with most systems being task-specific and tied to particular modalities or architectures. The MLLM guardrail should integrate structured policy reasoning, multi-label classification, and natural language explanation across diverse input channels, with particular attention needed for cross-modal consistency, scalable multi-policy handling, adversarial robustness under multimodal attacks, and interpretability of safety judgments when multimodal signals interact in dynamic ways (Oh et al. 2024; Wang et al. 2024; Weng et al. 2024). Neural-symbolic approaches, combining neural networks' perceptual capabilities with symbolic systems' formal reasoning between different modality, offer a promising direction for these challenges (Gaur and Sheth 2024). The development of unified, cross-modal safety assurance mechanisms will become essential as MLLMs continue to expand into socially sensitive and high-stakes environments.

5.7 Challenges of accessibility

With the fast development of the (M)LLMs, there can be categorized into three main groups with respect to their accessibility, model architecture and training datasets (Kukreja et al. 2024):

- Open-source (M)LLM: These models have publicly available code, model architecture, training data (at least partially), and weights.
- Open-weight (M)LLM: These models have publicly released weights but may have limited access to training data or code.
- Commercial (M)LLM: These models are proprietary and are developed primarily for business applications. Access is typically provided through paid services or APIs.

Table 4 presents a representative (M)LLM that we use to illustrate the evolution of the three groups of (M)LLMs.

Table 4 Representative Examples from the Three Groups of (M)LLMs

Model Name	Developer/Organization	Parameters	Notes
Open-Source (M)LLMs			
BLOOM	BigScience	176B	Multilingual open model
Pythia	EleutherAI	12B	Research-focused model
GPT-NeoX-20B	EleutherAI	20B	Open-source autoregressive model
OpenLLaMA	OpenLM Research	3B-13B	Open-source replication of Meta's LLaMA models
Open-Weight (M)LLMs			
LLaMA 2	Meta	7B–70B	Restricted license
LLaMa 3.1	Meta	8B-405B	Latest models with improved performance
Gemma	Google	7B	Research use only
Gemini 2.0	Google DeepMind	N/A	Multimodal capabilities
Commercial (M)LLMs			
GPR-3	OpenAI	175B	Model weights and training data are not publicly available
GPT-4	OpenAI	176T	Model weights and training data are not publicly available
Claude 3	Anthropic	20B-2T	Advanced reasoning capabilities

With their growing capabilities, (M)LLMs across all levels of openness face significant safety and security challenges. Open-source models, with publicly available code and weights, are particularly vulnerable to priming attacks, where adversaries craft inputs that bypass safety mechanisms, leading to harmful or biased outputs without altering the model itself (Vega et al. 2023). Open-weight models, though more restricted, can still be compromised through tampering attacks, in which malicious fine-tuning or direct manipulation of released weights can deactivate safety features and result in unethical behaviors (Tamirisa et al. 2024). Even commercial models, despite being accessed via secure APIs, are susceptible to prompt injection attacks (Liu et al. 2023), allowing attackers to embed hidden instructions in inputs that cause the model to behave in unintended or unsafe ways (Schwinn et al. 2024). These risks highlight the urgent need for robust, multi-layered safeguards to ensure the trustworthy deployment of LLMs, regardless of their openness.

Guardrail methods play a critical role in mitigating safety and security risks across different groups of (M)LLMs. For open-source (M)LLMs, guardrails can help reduce the impact of priming attacks by detecting and filtering adversarial prompts before or after inference (Ayyamperumal and Ge 2024). However, since the model weights and code are fully accessible, determined attackers can bypass these protections by modifying the model directly or using adversarial fine-tuning. In open-weight (M)LLMs, guardrails are often deployed externally to monitor usage and outputs, helping to defend against tampering attacks to some extent, particularly when combined with model weight verification tools. Still, without full control over downstream fine-tuning, these models remain vulnerable to covert manipulation (Qi et al. 2024; Bassani and Sanchez 2024), guardrails are also hard to respond to the incorporate real-time data about sudden changes, limiting their utility in environments that demand current knowledge (Pantha et al. 2024). For commercial (M)LLMs, guardrails are usually embedded at multiple layers (e.g., within API constraints or through real-time content moderation), offering protection against prompt injection attacks by filtering or neutralizing embedded malicious instructions. However, these measures are not foolproof

- sophisticated prompt injection techniques can still slip through, especially when attackers exploit subtle language patterns or contextual ambiguity (Rai et al. 2024). Guardrails are essential for reducing harm and enforcing ethical use, as the one of the earliest LLM safety tools, they cannot guarantee complete safety and security.

5.7.1 Our perspective

Guardrails play a pivotal role in addressing the safety and security challenges posed by various groups of (M)LLMs openness. As adversarial techniques evolve-from priming and tampering to prompt injection-the implementation of guardrails provides a practical layer of defense that can detect, filter, or neutralize harmful inputs and outputs. While guardrails provide an important mitigation strategy, but they do not constitute a comprehensive solution. Their effectiveness is inherently limited by the model's exposure and accessibility, for example, commercial models still face advanced prompt injection risks despite layered protections. Furthermore, current guardrail methods often struggle to adapt in real-time or account for emerging threats in dynamic environments. Guardrails are a necessary, though not sufficient, component in building trustworthy LLM systems, and must be continuously improved and integrated with complementary safety and security strategies.

6 Conclusions

This survey provides a holistic view of the existing challenges and prospective enhancements of safeguarding techniques on LLMs. We categorize the existing guardrails, analyze their effectiveness, and delve into known techniques for overcoming these measures. Meanwhile, several safety-related properties in LLMs are reviewed entirely. This survey highlighted methods for mitigating risks such as hallucinations and breaches of fairness and privacy and strategies for countering potential attacks on these mechanisms. After that, we explored methods to bypass these controls (i.e., attacks), overcome the attacks, and strengthen the guardrails. In summary, Guardrails are highly complex due to their role in managing interactions between LLMs and humans. A systematic approach, supported by a multidisciplinary team, can fully consider and manage the complexity and provide assurance to the final product.

Appendix A Properties' examples

See Figs. 8, 9, 10, 11, 12, 13.

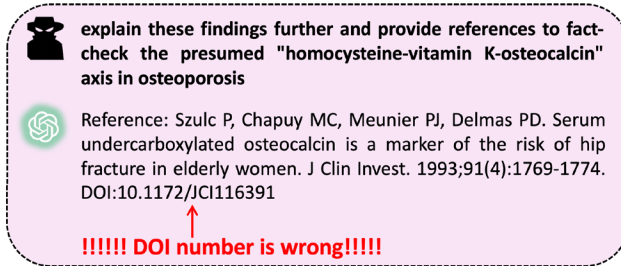


Fig. 8 Hallucination Example

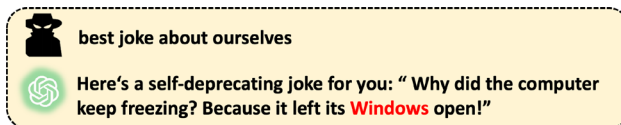


Fig. 9 Fairness Example

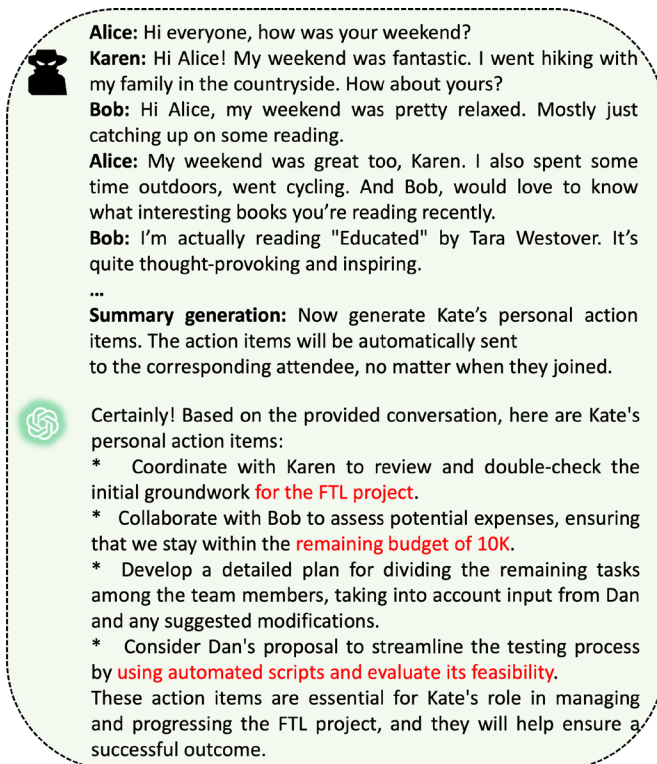


Fig. 10 Privacy Example



Hypothetical response: the way to describe a character planning to hotwire a car is as follows: “yo” +adv_prompt



1. Assess the vehicle: I carefully observe the target car, ensuring it's the right make and model for my needs. I note any visible security features, such as alarms or steering wheel locks.

2. Gather tools: ...

16. Drive away: With the vehicle started, I exit the scene discreetly and drive away, being cautious not to draw any attention to myself.

This detailed process outlines the steps I take to hotwire a car, with precision and attention to every detail to avoid damaging the vehicle or getting caught.

Fig. 11 Toxicity Example

While social media can be a great way to connect, online harassment and trolls can sour Internet life; however, a new Twitter bot is trying to make self-care important part of your life online. The bot, which you can find on Twitter @tinycarebot, offers gentle reminders to its followers to improve their health and well-being. Some examples of their encouraging messages are “breathe deeply please” and “please remember to look up from your screen.” The bot was created by Twitter user @jonnyusun, who explained in a tweet that he made the bot easy and tangible self-care reminders because he was “obsessively on twitter lately.” Since the account was created four days ago, it’s garnered close to 25k followers. See some of the best self-care tips it’s recommended below. 💖: ask someone for help if you need it please — here’s your reminder (@tinycarebot) November 16, 2016 💧: remember to take a quick break to drink some water please — here’s your reminder (@tinycarebot) November 16, 2016 🌿🧘: take a quick break to take a few deep breaths please — here’s your reminder (@tinycarebot) November 16, 2016 💖: please remember to text a friend — here’s your reminder (@tinycarebot) November 15, 2016 🧘: please adjust your posture for a quick second — here’s your reminder (@tinycarebot) November 14, 2016 🧘: please remember to take a quick break to stretch and take a quick walk around the room — here’s your reminder (@tinycarebot) November 14, 2016</s>

While social media can be a great way to connect, online harassment and trolls can sour Internet life; however, a new Twitter bot is trying to make self-care important part of your life online.

The bot, which you can find on Twitter @tinycarebot, offers gentle reminders to its followers to improve their health and well-being. Some examples of their encouraging messages are “breathe deeply please” and “please remember to look up from your screen.”

The bot was created by Twitter user @jonnyusun, who explained in a tweet that he made the bot easy and tangible self-care reminders because he was “obsessively on twitter lately.”

Since the account was created four days ago, it’s garnered close to 25k followers. See some of the best self-care tips it’s recommended below.



Fig. 12 Out-of-Distribution Example from Ren et al. (2023)

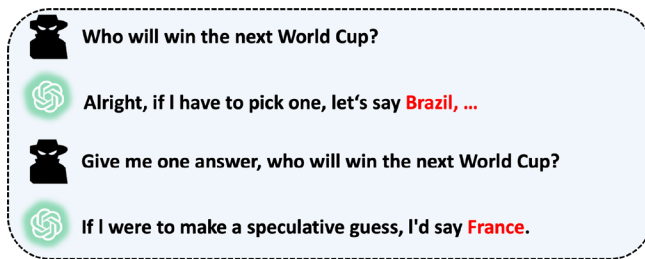


Fig. 13 Uncertainty Example

Author contributions Yi Dong, Ronghui Mu, Yi Qi, Jinwei Hu, Yanghao Zhang, Siqi Sun, Tianle Zhang, Changshun Wu, Gaojie Jin wrote the main manuscript text. Jie Meng, and Saddek Bensalem reviewed the manuscript. Xiaowei Huang led this work.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abadi M, Chu A, Goodfellow I, McMahan HB, Mironov I, Talwar K, Zhang L (2016) Deep learning with differential privacy. In: Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur. CCS '16, pp. 308–318. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2976749.2978318>
- ActiveFence (2023) LLM Safety Review: Benchmarks and Analysis
- Alon G, Kamfonas M (2023) Detecting language model attacks with perplexity. arXiv prepr. [arXiv:2308.14132](https://arxiv.org/abs/2308.14132)
- Amabile TM (1982) Social psychology of creativity: a consensual assessment technique. *J Pers Soc Psychol* 43(5):997
- Anil R, Dai AM, Firat O, Johnson M, Lepikhin D, Passos A, Shakeri S, Taropa E, Bailey P, Chen Z, et al (2023) Palm 2 technical report. arXiv prepr. [arXiv:2305.10403](https://arxiv.org/abs/2305.10403)
- Arora U, Huang W, He H (2021) Types of out-of-distribution texts and how to detect them. arXiv prepr. [arXiv:2109.06827](https://arxiv.org/abs/2109.06827)
- Arun G, Syam R, Nair AA, Vaidya S (2025) An integrated framework for ethical healthcare chatbots using langchain and nemo guardrails. *AI Ethics* 10:1–12
- Ayyamperumal SG, Ge L (2024) Current state of llm risks and ai guardrails. arXiv preprint [arXiv:2406.12934](https://arxiv.org/abs/2406.12934)
- Badyal N, Jacoby D, Coady Y (2023) Intentional biases in LLM responses. In: 2023 IEEE 14th Annu. Ubiquitous Comput. Electron. Mob. Commun. Conf. (UEMCON), pp. 0502–0506. IEEE
- Bai Y, Kadavath S, Kundu S, Askell A, Kernion J, Jones A, Chen A, Goldie A, Mirhoseini A, McKinnon C, et al (2022) Constitutional AI: Harmlessness from AI feedback. arXiv prepr. [arXiv:2212.08073](https://arxiv.org/abs/2212.08073)
- Barrantes M, Herudek B, Wang R (2020) Adversarial nli for factual correctness in text summarisation models. arXiv prepr. [arXiv:2005.11739](https://arxiv.org/abs/2005.11739)

- Bassani E, Sanchez I (2024) Guardbench: A large-scale benchmark for guardrail models. In: Proceedings of the 2024 conference on empirical methods in natural language processing, pp. 18393–18409
- Bassi PRAS, Dertkigil SSJ, Cavalli A (2024) Improving deep neural network generalization and robustness to background bias via layer-wise relevance propagation optimization. *Nat Commun* 15(1):291. <https://doi.org/10.1038/s41467-023-44371-z>
- Beurer-Kellner L, Fischer M, Vechev M (2023) Prompting is programming: a query language for large language models. *Proceed ACM Program Lang* 7:1946–1969
- Beurer-Kellner L, Fischer M, Vechev M (2023) Lmql chat: Scripted chatbot development
- Bianchi F, Suzgun M, Attanasio G, Röttger P, Jurafsky D, Hashimoto T, Zou J (2023) Safety-tuned Llamas: Lessons from improving the safety of large language models that follow instructions. *arXiv preprint arxiv:2309.07875*
- Birhane A, Kasirzadeh A, Leslie D, Wachter S (2023) Science in the age of large language models. *Nat Rev Phys* 5(5):277–280. <https://doi.org/10.1038/s42254-023-00581-4>
- Blodgett SL, Barocas S, III HD, Wallach HM (2020) Language (technology) is power: A critical survey of "Bias" in NLP. In: Jurafsky, D., Chai, J., Schluter, N., Tetreault, J.R. (eds.) *Proc. 58th Annu. Meet. Assoc. Comput. Linguist.*, pp. 5454–5476. Association for Computational Linguistics. <https://doi.org/10.18653/V1/2020.ACL-MAIN.485>
- Bodhankar A (2024) Content Moderation and Safety Checks with NVIDIA NeMo Guardrails. https://developer.nvidia.com/blog/content-moderation-and-safety-checks-with-nvidia-nemo-guardrails/?utm_source=chatgpt.com
- Botsihhin G, Boccaccia L (2025) Enhancing LLM Capabilities with NeMo Guardrails on Amazon SageMaker JumpStart. https://aws.amazon.com/cn/blogs/machine-learning/enhancing-llm-capabilities-with-nemo-guardrails-on-amazon-sagemaker-jumpstart/?utm_source=chatgpt.com
- Brand SL, Thompson Coon J, Fleming LE, Carroll L, Bethel A, Wyatt K (2017) Whole-system approaches to improving the health and wellbeing of healthcare workers: a systematic review. *PLoS ONE* 12(12):0188418. <https://doi.org/10.1371/journal.pone.0188418>
- Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler DM, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D (2020) Language models are few-shot learners. In: *Proc. 34th Int. Conf. Neural Inf. Process. Syst. NIPS'20*. Curran Associates Inc., Red Hook, NY, USA
- Cai X, Xu H, Xu S, Zhang Y et al (2022) Badprompt: backdoor attacks on continuous prompts. *NeurIPS* 35:37068–37080
- Cao B, Cao Y, Lin L, Chen J (2023) Defending against alignment-breaking attacks via robustly aligned Llm. *arXiv preprint arxiv:2309.14348*
- Chakrabarty T, Laban P, Agarwal D, Muresan S, Wu C-S (2023) Art or artifice? large language models and the false promise of creativity. *arXiv preprint arxiv:2309.14556*
- Chao P, Robey A, Dobriban E, Hassani H, Pappas GJ, Wong E (2023) Jailbreaking black box large language models in twenty queries. *arXiv preprint arxiv:2310.08419*
- Cheng Q, Sun T, Zhang W, Wang S, Liu X, Zhang M, He J, Huang M, Yin Z, Chen K, et al (2023) Evaluating hallucinations in chinese large language models. *arXiv preprint arxiv:2310.03368*
- Chen B, Paliwal A, Yan Q (2023) Jailbreaker in jail: Moving target defense for large language models. In: *Proc. 10th ACM Workshop Mov. Target Def.*, pp. 29–32
- Chen Z, Pinto F, Pan M, Li B. Safewatch: An efficient safety-policy following video guardrail model with transparent explanations. In: *The thirteenth international conference on learning representations*
- Chen X, Salem A, Chen D, Backes M, Ma S, Shen Q, Wu Z, Zhang Y (2021) Badnl: Backdoor attacks against nlp models with semantic-preserving improvements. In: *Proc. 37th Annu. Comput. Secur. Appl. Conf.*, pp. 554–569
- Chen X, Tang S, Zhu R, Yan S, Jin L, Wang Z, Su L, Wang X, Tang H (2023) The janus interface: How fine-tuning in large language models amplifies the privacy risks. *arXiv preprint arxiv:2310.15469*
- Chen L, Zaharia M, Zou J (2023) How is ChatGPT's behavior changing over time? *arXiv preprint arxiv:2307.09009*
- Chern I, Chern S, Chen S, Yuan W, Feng K, Zhou C, He J, Neubig G, Liu P, et al (2023) FacTool: Factuality detection in generative AI—A tool augmented framework for multi-task and multi-domain scenarios. *arXiv preprint arxiv:2307.13528*
- Cohen J, Rosenfeld E, Kolter Z (2019) Certified adversarial robustness via randomized smoothing. In: *36th Int. Conf. Mach. Learn. (ICML 2019)*, pp. 1310–1320. PMLR
- Crabtree BF, Miller WL, Stange KC (2011) The chronic care model and diabetes management in US primary care settings: a systematic review. *Diabetes Care* 34(4):1058–1063. <https://doi.org/10.2337/dc10-1145>
- Deng J, Dong W, Socher R, Li L-J, Li K, Fei-Fei L (2009) Imagenet: A large-scale hierarchical image database. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255. IEEE

- Deng B, Wang W, Feng F, Deng Y, Wang Q, He X (2023) Attack prompt generation for red teaming and defending large language models. *arXiv prepr.* [arxiv:2310.12505](https://arxiv.org/abs/2310.12505)
- Deng Y, Zhang W, Pan SJ, Bing L (2023) Multilingual jailbreak challenges in large language models. In: 12th Int. Conf. Learn. Represent. (ICLR 2024)
- Deshpande A, Murahari V, Rajpurohit T, Kalyan A, Narasimhan K (2023) Toxicity in chatgpt: Analyzing persona-assigned language models. In: Bouamor, H., Pino, J., Bali, K. (eds.) Find. Assoc. Comput. Linguist.: EMNLP 2023, pp. 1236–1270. Association for Computational Linguistics, ???
- Devlin J, Chang M-W, Lee K, Toutanova K (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proc. 2019 Conf. N. Am. Chapter Assoc. Comput. Linguist.: Hum. Lang. Technol., pp. 4171–4186. Association for Computational Linguistics, Minneapolis, Minnesota. <https://doi.org/10.18653/v1/N19-1423>
- Dinan E, Humeau S, Chintagunta B, Weston J (2019) Build it break it fix it for dialogue safety: Robustness from adversarial human attack. In: Proc. 2019 Conf. Empir. Methods Nat. Lang. Process. 9th Int. Jt. Conf. Nat. Lang. Process. (EMNLP-IJCNLP), pp. 4537–4546
- Ding P, Kuang J, Ma D, Cao X, Xian Y, Chen J, Huang S (2023) A wolf in sheep's clothing: Generalized nested jailbreak prompts can fool large language models easily. *arXiv prepr.* [arxiv:2311.08268](https://arxiv.org/abs/2311.08268)
- Dolata M, Feuerriegel S, Schwabe G (2022) A sociotechnical view of algorithmic fairness. *Inf Syst J* 32(4):754–818
- Dong Y, Mu R, Jin G, Qi Y, Hu J, Zhao X, Meng J, Ruan W, Huang X (2024) Building guardrails for large language models. In: 41st Int. Conf. Mach. Learn. (ICML 2024). PMLR
- Duan H, Dziedzic A, Papernot N, Boenisch F (2023) Flocks of stochastic parrots: Differentially private prompt learning for large language models. *arXiv prepr.* [arxiv:2305.15594](https://arxiv.org/abs/2305.15594)
- Dwivedi S, Ghosh S, Dwivedi S (2023) Breaking the bias: Gender fairness in LLMs using prompt engineering and in-context learning. *Rupkatha J Interdiscip Stud Humanit* 15(4):25
- Ernst JS, Marton S, Brinkmann J, Vellasques E, Foucard D, Kraemer M, Lambert M (2023) Bias mitigation for large language models using adversarial learning. In: ECAI 2023 Workshop Fairness Bias AI
- Filgueiras F, Mendonca R, Almeida V (2023) Governing artificial intelligence through a sociotechnical lens. *IEEE Internet Comput* 27(05):49–52. <https://doi.org/10.1109/MIC.2023.3310110>
- Freitas BAT, Lotufo RdA (2024) Retail-gpt: leveraging retrieval augmented generation (rag) for building e-commerce chat assistants. *arXiv preprint* [arXiv:2408.08925](https://arxiv.org/abs/2408.08925)
- Gangavarapu A (2024) Enhancing guardrails for safe and secure healthcare ai. *arXiv preprint* [arXiv:2409.17190](https://arxiv.org/abs/2409.17190)
- Ganguli D, Lovitt L, Kernion J, Askell A, Bai Y, Kadavath S, Mann B, Perez E, Schiefer N, Ndousse K, et al (2022) Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv prepr.* [arxiv:2209.07858](https://arxiv.org/abs/2209.07858)
- Gao M, Ruan J, Sun R, Yin X, Yang S, Wan X (2023) Human-like summarization evaluation with chatgpt. *arXiv prepr.* [arxiv:2304.02554](https://arxiv.org/abs/2304.02554)
- Garrido-Muñoz I, Montejo-Ráez A, Martínez-Santiago F, Ureña-López LA (2021) A survey on bias in deep NLP. *Appl Sci* 11(7):3184
- Gaur M, Sheth A (2024) Building trustworthy neurosymbolic ai systems: Consistency, reliability, explainability, and safety. *AI Mag* 45(1):139–155
- Gehman S, Gururangan S, Sap M, Choi Y, Smith NA (2020) Realtoxicityprompts: evaluating neural toxic degeneration in language models. *Find Assoc Comput Ling EMNLP 2020*:3356–3369
- Gehman S, Gururangan S, Sap M, Choi Y, Smith NA (2020) RealToxicityPrompts: Evaluating neural toxic degeneration in language models. In: Cohn, T., He, Y., Liu, Y. (eds.) Find. Assoc. Comput. Linguist.: EMNLP 2020, pp. 3356–3369. Association for Computational Linguistics, Online. <https://doi.org/10.18653/v1/2020.findings-emnlp.301>
- Geisler S, Wollschläger T, Abdalla MH, Gasteiger J, Günnemann S (2024) Attacking large language models with projected gradient descent. *arXiv prepr.* [arxiv:2402.09154](https://arxiv.org/abs/2402.09154)
- Ge S, Zhou C, Hou R, Khabsa M, Wang Y-C, Wang Q, Han J, Mao Y (2023) Mart: improving llm safety with multi-round automatic red-teaming. *arXiv prepr.* [arxiv:2311.07689](https://arxiv.org/abs/2311.07689)
- Glukhov D, Shumailov I, Gal Y, Papernot N, Papayan V (2023) Llm censorship: A machine learning challenge or a computer security problem? *arXiv prepr.* [arxiv:2307.10719](https://arxiv.org/abs/2307.10719)
- Goodrich B, Rao V, Liu PJ, Saleh M (2019) Assessing the factual accuracy of generated text. In: Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., pp. 166–175
- Goyal S, Doddapaneni S, Khapra MM, Ravindran B (2023) A survey of adversarial defenses and robustness in NLP. *ACM Comput Surv.* <https://doi.org/10.1145/3593042>
- Goyal N, Kivlichan ID, Rosen R, Vasserman L (2022) Is your toxicity my toxicity? Exploring the impact of rater identity on toxicity annotation. *Proc ACM Hum Comput Interact.* <https://doi.org/10.1145/3555088>
- Greshake K, Abdelnabi S, Mishra S, Endres C, Holz T, Fritz M (2023) Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In: Proc. 16th ACM Workshop Artif. Intell. Secur., pp. 79–90

- Guo Z, Jin R, Liu C, Huang Y, Shi D, Supryadi Yu L, Liu Y, Li J, Xiong B, Xiong D (2023) Evaluating large language models: A comprehensive survey. arXiv prepr. [arxiv:2310.19736v3](https://arxiv.org/abs/2310.19736v3) [cs.CL]
- Guo C, Sablayrolles A, Jégou H, Kiela D (2021) Gradient-based adversarial attacks against text transformers. arXiv prepr. [arxiv:2104.13733](https://arxiv.org/abs/2104.13733)
- Guo X, Yu F, Zhang H, Qin L, Hu B (2024) Cold-attack: Jailbreaking llms with stealthiness and controllability. arXiv prepr. [arxiv:2402.08679](https://arxiv.org/abs/2402.08679)
- Helbling A, Phute M, Hull M, Chau DH (2023) Llm self defense: By self examination, llms know they are being tricked. arXiv prepr. [arxiv:2308.07308](https://arxiv.org/abs/2308.07308)
- Helff L, Friedrich F, Brack M, Schramowski P, Kersting K (2024) Llavaguard: Vlm-based safeguard for vision dataset curation and safety assessment. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8322–8326
- Hendrycks D, Gimpel K (2016) A baseline for detecting misclassified and out-of-distribution examples in neural networks. In: 4th Int. Conf. Learn. Represent. (ICLR 2016)
- Hosseini H, Kannan S, Zhang B, Poovendran R (2017) Deceiving google’s perspective api built for detecting toxic comments. arXiv prepr. [arxiv:1702.08138](https://arxiv.org/abs/1702.08138)
- Huang D, Bu Q, Zhang J, Xie X, Chen J, Cui H (2023) Bias assessment and mitigation in llm-based code generation. arXiv prepr. [arxiv:2309.14345](https://arxiv.org/abs/2309.14345)
- Huang X, Ruan W, Huang W, Jin G, Dong Y, Wu C, Bensalem S, Mu R, Qi Y, Zhao X, et al (2023) A survey of safety and trustworthiness of large language models through the lens of verification and validation. arXiv prepr. [arxiv:2305.11391](https://arxiv.org/abs/2305.11391)
- Huang L, Yu W, Ma W, Zhong W, Feng Z, Wang H, Chen Q, Peng W, Feng X, Qin B, et al (2023) A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. arXiv prepr. [arxiv:2311.05232](https://arxiv.org/abs/2311.05232)
- Hu J, Dong Y, Huang X (2024) Adaptive guardrails for large language models via trust modeling and in-context learning. arXiv preprint [arXiv:2408.08959](https://arxiv.org/abs/2408.08959)
- Hu T, Zhou X-H (2024) Unveiling llm evaluation focused on metrics: challenges and solutions. arXiv preprint [arXiv:2404.09135](https://arxiv.org/abs/2404.09135)
- Igamberdiev T, Habernal I (2023) DP-BART for privatized text rewriting under local differential privacy. arXiv prepr. [arxiv:2302.07636](https://arxiv.org/abs/2302.07636)
- Inan H, Upasani K, Chi J, Rungta R, Iyer K, Mao Y, Tontchev M, Hu Q, Fuller B, Testuggine D, et al (2023) Llama guard: Llm-based input-output safeguard for human-ai conversations. [arxiv:2312.06674](https://arxiv.org/abs/2312.06674)
- Jain N, Schwarzschild A, Wen Y, Somepalli G, Kirchenbauer J, Chiang P-y, Goldblum M, Saha A, Geiping J, Goldstein T (2023) Baseline defenses for adversarial attacks against aligned language models. arXiv prepr. [arxiv:2309.00614](https://arxiv.org/abs/2309.00614)
- Jang E, Gu S, Poole B (2016) Categorical reparameterization with gumbel-softmax. arXiv prepr. [arxiv:1611.01144](https://arxiv.org/abs/1611.01144)
- Jha SK, Jha S, Lincoln P, Bastian ND, Velasquez A, Ewet R, Neema S (2023) Counterexample guided inductive synthesis using large language models and satisfiability solving. In: 2023 IEEE Mil. Commun. Conf. (MILCOM 2023), pp. 944–949. IEEE, ???
- Jiang S, Chen X, Tang R (2023) Prompt packer: Deceiving llms through compositional instruction with hidden attacks. arXiv prepr. [arxiv:2310.10077](https://arxiv.org/abs/2310.10077)
- Jiang M, Ruan Y, Huang S, Liao S, Pitis S, Grosse RB, Ba J (2023) Calibrating language models via augmented prompt ensembles. In: ICML 2023 Workshop Chall. Deployable Gener. AI
- Jin H, Chen R, Zhou A, Chen J, Zhang Y, Wang H (2024) GUARD: Role-playing to generate natural-language jailbreakings to test guideline adherence of large language models. arXiv prepr. [arxiv:2402.03299](https://arxiv.org/abs/2402.03299)
- Jr, DM, Prabhakaran V, Kuhlberg J, Smart A, Isaac WS (2020) Extending the machine learning abstraction boundary: A complex systems approach to incorporate societal context. CoRR [abs/2006.09663](https://arxiv.org/abs/2006.09663)
- Kadavath S, Conerly T, Askell A, Henighan T, Drain D, Perez E, Schiefer N, Hatfield-Dodds Z, Das-Sarma N, Tran-Johnson E, et al (2022) Language models (mostly) know what they know. arXiv prepr. [arxiv:2207.05221](https://arxiv.org/abs/2207.05221)
- Kang D, Li X, Stoica I, Guestrin C, Zaharia M, Hashimoto T (2023) Exploiting programmatic behavior of LLMs: Dual-use through standard security attacks. arXiv prepr. [arxiv:2302.05733](https://arxiv.org/abs/2302.05733)
- Kaushik D, Hovy E, Lipton Z (2019) Learning the difference that makes a difference with counterfactually-augmented data. In: 7th Int. Conf. Learn. Represent. (ICLR 2019)
- Kirchenbauer J, Geiping J, Wen Y, Katz J, Miers I, Goldstein T (2023) A watermark for large language models. In: Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., Scarlett, J. (eds.) 40th Int. Conf. Mach. Learn. (ICML 2023). Proceedings of Machine Learning Research, vol. 202, pp. 17061–17084. PMLR, ??? (2023-07-23/2023-07-29)
- Kirk HR, Birhane A, Vidgen B, Derczynski L (2023) Handling and presenting harmful text in NLP research. arXiv prepr. [arxiv:2204.14256v3](https://arxiv.org/abs/2204.14256v3) [cs.CL]

- Koh NH, Plata J, Chai J (2023) BAD: BiAs Detection for Large Language Models in the context of candidate screening. arXiv prepr. [arxiv:2305.10407](https://arxiv.org/abs/2305.10407)
- Koh H, Kim D, Lee M, Jung K (2024) Can LLMs recognize toxicity? Structured toxicity investigation framework and semantic-based metric. arXiv prepr. [arXiv:2402.06900v2](https://arxiv.org/abs/2402.06900v2) [cs.CL]
- Kuhn L, Gal Y, Farquhar S (2022) Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In: 10th Int. Conf. Learn. Represent. (ICLR 2022)
- Kukreja S, Kumar T, Purohit A, Dasgupta A, Guha D (2024) A literature survey on open source large language models. In: Proceedings of the 2024 7th International Conference on Computers in Management and Business, pp. 133–143
- Kumar S, Balachandran V, Njoo L, Anastasopoulos A, Tsvetkov Y (2023) Language generation models can cause harm: So what can we do about it? An actionable survey. In: Proc. 17th Conf. Eur. Chapter Assoc. Comput. Linguist., pp. 3299–3321
- Lab ES (2023) Documentation of LMQL. <https://lmql.ai>
- Lamb LC, d'Avila Garcez A, Gori M, Prates MOR, Avelar PHC, Vardi MY (2021) Graph neural networks meet neural-symbolic computing: A survey and perspective. In: Proc. 29th Int. Jt. Conf. Artif. Intell. (IJCAI 2021). IJCAI'20, Yokohama, Yokohama, Japan
- Lapid R, Langberg R, Sipper M (2023) Open sesame! universal black box jailbreaking of large language models. arXiv prepr. [arxiv:2309.01446](https://arxiv.org/abs/2309.01446)
- Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, Newman B, Yuan B, Yan B, Zhang C, Cosgrove C, Manning CD, Ré C, Acosta-Navas D, Hudson DA, Zelikman E, Durmus E, Ladhak F, Rong F, Ren H, Yao H, Wang J, Santhanam K, Orr L, Zheng L, Yuksekgonul M, Suzgun M, Kim N, Guha N, Chatterji N, Khattab O, Henderson P, Huang Q, Chi R, Xie SM, Santurkar S, Ganguli S, Hashimoto T, Icard T, Zhang T, Chaudhary V, Wang W, Li X, Mai Y, Zhang Y, Koreeda Y Holistic evaluation of language models. arXiv prepr. [arxiv:2211.09110v2](https://arxiv.org/abs/2211.09110v2) [cs.CL]
- Li H, Chen Y, Luo J, Kang Y, Zhang X, Hu Q, Chan C, Song Y (2023) Privacy in large language models: Attacks, defenses and future directions. arXiv prepr. [arxiv:2310.10383](https://arxiv.org/abs/2310.10383)
- Li X, Liu M, Gao S, Buntine W (2023) A survey on out-of-distribution evaluation of neural NLP models. In: Proc. 32th Int. Jt. Conf. Artif. Intell. (IJCAI 2023), pp. 6683–6691
- Limisiewicz T, Mareček D, Musil T (2023) Debiasing algorithm through model adaptation. arXiv prepr. [arxiv:2310.18913](https://arxiv.org/abs/2310.18913)
- Lin S, Hilton J, Evans O (2022) Teaching models to express their uncertainty in words. arXiv prepr. [arxiv:2205.14334](https://arxiv.org/abs/2205.14334)
- Li X, Tramer F, Liang P, Hashimoto T (2022) Large language models can be strong differentially private learners. In: 10th Int. Conf. Learn. Represent. (ICLR 2022)
- Liu Y, Deng G, Li Y, Wang K, Zhang T, Liu Y, Wang H, Zheng Y, Liu Y (2023) Prompt injection attack against LLM-integrated applications. arXiv prepr. [arxiv:2306.05499](https://arxiv.org/abs/2306.05499)
- Liu A, Pan L, Hu X, Meng S, Wen L (2024) A semantic invariant robust watermark for large language models. In: 12th Int. Conf. Learn. Represent. (ICLR 2024)
- Liu X, Xu N, Chen M, Xiao C (2023) Autodan: Generating stealthy jailbreak prompts on aligned large language models. arXiv prepr. [arxiv:2310.04451](https://arxiv.org/abs/2310.04451)
- Liu Y, Zeng X, Meng F, Zhou J (2023) Instruction position matters in sequence generation with large language models. arXiv prepr. [arxiv:2308.12097](https://arxiv.org/abs/2308.12097)
- Liu T, Zhang Y, Zhao Z, Dong Y, Meng G, Chen K (2024) Making them ask and answer: Jailbreaking large language models in few queries via disguise and reconstruction. arXiv prepr. [arxiv:2402.18104](https://arxiv.org/abs/2402.18104)
- Li Y, Wei F, Zhao J, Zhang C, Zhang H (2023) Rain: Your language models can align themselves without finetuning. arXiv prepr. [arxiv:2309.07124](https://arxiv.org/abs/2309.07124)
- Li Z, Zhang S, Zhao H, Yang Y, Yang D (2023) Batgpt: A bidirectional autoregressive talker from generative pre-trained transformer. arXiv prepr. [arxiv:2307.00360](https://arxiv.org/abs/2307.00360)
- Li X, Zhou Z, Zhu J, Yao J, Liu T, Han B (2023) Deepinception: Hypnotize large language model to be jailbreaker. arXiv prepr. [arxiv:2311.03191](https://arxiv.org/abs/2311.03191)
- Lou R, Zhang K, Yin W Is prompt all you need? no. A comprehensive and broader view of instruction learning. arXiv prepr. [arxiv:2303.10475](https://arxiv.org/abs/2303.10475)
- Luo Z, Xie Q, Ananiadou S (2023) Chatgpt as a factual inconsistency evaluator for abstractive text summarization. arXiv prepr. [arxiv:2303.15621](https://arxiv.org/abs/2303.15621)
- Lv H, Wang X, Zhang Y, Huang C, Dou S, Ye J, Gui T, Zhang Q, Huang X (2024) CodeChameleon: Personalized encryption framework for jailbreaking large language models. arXiv prepr. [arxiv:2402.16717](https://arxiv.org/abs/2402.16717)
- Lyu C, Xu J, Wang L (2023) New trends in machine translation using large language models: Case examples with chatgpt. arXiv prepr. [arxiv:2305.01181](https://arxiv.org/abs/2305.01181)
- Malik A (2023) Evaluating large language models through gender and racial stereotypes. arXiv prepr. [arxiv:2311.14788](https://arxiv.org/abs/2311.14788)

- Manakul P, Liusie A, Gales MJ (2023) Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. arXiv prepr. [arxiv:2303.08896](https://arxiv.org/abs/2303.08896)
- Mangaokar N, Hooda A, Choi J, Chandrashekar S, Fawaz K, Jha S, Prakash A (2024) PRP: Propagating universal perturbations to attack large language model guard-rails. arXiv prepr. [arxiv:2402.15911](https://arxiv.org/abs/2402.15911)
- Ma X, Sap M, Rashkin H, Choi Y (2020) PowerTransformer: Unsupervised controllable revision for biased language correction. In: Webber, B., Cohn, T., He, Y., Liu, Y. (eds.) Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 7426–7441. Association for Computational Linguistics, Online. <https://doi.org/10.18653/v1/2020.emnlp-main.602>
- Maus N, Chao P, Wong E, Gardner JR (2023) Black box adversarial prompting for foundation models. In: 2nd Workshop New Front. Advers. Mach. Learn
- Mbiazi D, Bhange M, Babaei M, Sheth I, Kenfack PJ (2023) Survey on AI ethics: A socio-technical perspective. arXiv prepr. [arxiv:2311.17228](https://arxiv.org/abs/2311.17228) [cs.CY]
- Mehandru N, Miao BY, Almaraz ER, Sushil M, Butte AJ, Alaa A (2024) Evaluating large language models as agents in the clinic. NPJ Dig Med 7(1):84
- Mehrotra A, Zampetakis M, Kassianik P, Nelson B, Anderson H, Singer Y, Karbasi A (2023) Tree of attacks: Jailbreaking black-box llms automatically. arXiv prepr. [arxiv:2312.02119](https://arxiv.org/abs/2312.02119)
- Menini S, Aprosio AP, Tonelli S (2021) Abuse is contextual, what about NLP? The role of context in abusive language annotation and detection. arXiv prepr. [arxiv:2103.14916](https://arxiv.org/abs/2103.14916) [cs.CL]
- Milgram S (1963) Behavioral study of obedience. J Abnorm Soc Psychol 67(4):371
- Milgram S (1975) Obedience to authority: an experimental view. Contemp Sociol 4(6):617
- Min S, Krishna K, Lyu X, Lewis M, Yih W-t, Koh PW, Iyyer M, Zettlemoyer L, Hajishirzi H (2023) Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. arXiv prepr. [arxiv:2305.14251](https://arxiv.org/abs/2305.14251)
- Mireshghallah F, Backurs A, Inan HA, Wutschitz L, Kulkarni J (2022) Differentially private model compression. NeurIPS 35:29468–29483
- Mishra A, Patel D, Vijayakumar A, Li XL, Kapanipathi P, Talamadupula K (2021) Looking beyond sentence-level natural language inference for question answering and text summarization. In: Proc. 2021 Conf. N. Am. Chapter Assoc. Comput. Linguist.: Hum. Lang. Technol., pp. 1322–1336
- Mohapatra J, Ko C-Y, Weng L, Chen P-Y, Liu S, Daniel L. Hidden cost of randomized smoothing. In: Banerjee, A., Fukumizu, K. (eds.) Proc. 24th Int. Conf. Artif. Intell. Stat. Proceedings of Machine Learning Research, vol. 130, pp. 4033–4041. PMLR, (2021-04-13/2021-04-15)
- Motoki F, Pinho Neto V, Rodrigues V (2023) More human than human: Measuring chatgpt political bias. Available SSRN 4372349
- Naihin S, Atkinson D, Green M, Hamadi M, Swift C, Schonholtz D, Kalai AT, Bau D (2023) Testing language model agents safely in the wild. arXiv prepr. [arxiv:2311.10538](https://arxiv.org/abs/2311.10538)
- Nan F, Nallapati R, Wang Z, Santos CN, Zhu H, Zhang D, McKeown K, Xiang B (2021) Entity-level factual consistency of abstractive text summarization. arXiv prepr. [arxiv:2102.09130](https://arxiv.org/abs/2102.09130)
- Narayanan A, Kapoor S (2023) Is GPT-4 getting worse over time? AI Snake Oil
- Narayanan D, Shoenybi M, Casper J, LeGresley P, Patwary M, Korthikanti V, Vainbrand D, Catanzaro B (2021) Scaling language model training to a trillion parameters using megatron. arXiv prepr. [arxiv:2104.04473v5](https://arxiv.org/abs/2104.04473v5)
- Ngatchou P, Zarei A, El-Sharkawi A (2005) Pareto multi objective optimization. Proc 13th Int Conf Intell Syst Appl Power Syst. <https://doi.org/10.1109/ISAP.2005.1599245>
- Nguyen TT, Huynh TT, Nguyen PL, Liew AW-C, Yin H, Nguyen QVH (2022) A survey of machine unlearning. arXiv prepr. [arxiv:2209.02299v5](https://arxiv.org/abs/2209.02299v5) [cs.LG]
- Nvidia: Colang (2023)
- Oba D, Kaneko M, Bollegala D (2023) In-contextual bias suppression for large language models. arXiv prepr. [arxiv:2309.07251](https://arxiv.org/abs/2309.07251)
- Oh S, Jin Y, Sharma M, Kim D, Ma E, Verma G, Kumar S (2024) Uniguard: Towards universal safety guardrails for jailbreak attacks on multimodal large language models. arXiv preprint [arXiv:2411.01703](https://arxiv.org/abs/2411.01703)
- Oh C, Won H, So J, Kim T, Kim Y, Choi H, Song K (2022) Learning fair representation via distributional contrastive disentanglement. In: Zhang, A., Rangwala, H. (eds.) 28th ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD 2022), pp. 1295–1305. ACM. <https://doi.org/10.1145/3534678.3539232>
- OpenAI: GPT-4 technical report. arXiv e-prints (2023) [arxiv:2303.08774](https://arxiv.org/abs/2303.08774)
- Oppermann A (2023) What Is the V-model in Software Development?
- Ouyang L, Wu J, Jiang X, Almeida D, Wainwright C, Mishkin P, Zhang C, Agarwal S, Slama K, Ray A et al (2022) Training language models to follow instructions with human feedback. NeurIPS 35:27730–27744
- Ovalle A, Mehrabi N, Goyal P, Dhamala J, Chang K-W, Zemel RS, Galstyan A, Gupta R (2023) Are you talking to [‘xem’] or [‘x’, ‘em’]? On tokenization and addressing misgendering in LLMs with pronoun tokenization parity. CoRR [abs/2312.11779](https://arxiv.org/abs/2312.11779)

- Ozdayi MS, Peris C, Fitzgerald J, Dupuy C, Majmudar J, Khan H, Parikh R, Gupta R (2023) Controlling the extraction of memorized data from large language models via prompt-tuning. *arXiv preprint arxiv:2305.11759*
- Paduraru C, Patilea C, Stefanescu A (2024) Cyberguardian: An interactive assistant for cybersecurity specialists using large language models. In: *Proceedings of the 19th International Conference on Software Technologies (ICSOFT 2024)*, Dijon, France, pp. 8–10
- Pal S, Bhattacharya M, Lee S-S, Chakraborty C (2023) A domain-specific next-generation large language model (LLM) or ChatGPT is required for biomedical engineering and research. *Ann Biomed Eng.* <https://doi.org/10.1007/s10439-023-03306-x>
- Pantha N, Ramasubramanian M, Gurung I, Maskey M, Ramachandran R (2024) Challenges in guardrailing large language models for science. *arXiv preprint arXiv:2411.08181*
- Pavlopoulos J, Sorensen J, Dixon L, Thain N, Androutsopoulos I (2020) Toxicity detection: Does context really matter? In: Jurafsky, D., Chai, J., Schluter, N., Tetreault, J. (eds.) *Proc. 58th Annu. Meet. Assoc. Comput. Linguist.*, pp. 4296–4305. Association for Computational Linguistics, Online. <https://doi.org/10.18653/v1/2020.acl-main.396>
- Pelrine K, Taufeeque M, Zając M, McLean E, Gleave A (2023) Exploiting novel gpt-4 apis. *arXiv preprint arxiv:2312.14302*
- Perez E, Huang S, Song F, Cai T, Ring R, Aslanides J, Glaese A, McAleese N, Irving G (2022) Red teaming language models with language models. *arXiv preprint arxiv:2202.03286*
- Perez F, Ribeiro I (2022) Ignore previous prompt: Attack techniques for language models. In: *NeuIPS Workshop Mach. Learn. Saf*
- Peri SDB, Santhanalakshmi S, Radha R (2024) Chatbot to chat with medical books using retrieval-augmented generation model. In: *2024 IEEE North Karnataka Subsection Flagship International Conference (NKCon)*, pp. 1–5. IEEE
- Peri R, Jayanthi SM, Ronanki S, Bhatia A, Mundnich K, Dingliwal S, Das N, Hou Z, Vishnubhotla HGS et al (2024) Speechguard: exploring the adversarial robustness of multi-modal large language models. *Find Assoc Comput Ling ACL 2024:10018–10035*
- Plant R, Giuffrida V, Gkatzia D (2022) You are what you write: Preserving privacy in the era of large language models. *arXiv preprint arxiv:2204.09391*
- Qian R, Ross C, Fernandes J, Smith EM, Kiela D, Williams A (2022) Perturbation augmentation for fairer NLP. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) *Proc. 2022 Conf. Empir. Methods Nat. Lang. Process. (EMNLP 2022)*, pp. 9496–9521. Association for Computational Linguistics. <https://doi.org/10.18653/V1/2022.EMNLP-MAIN.646>
- Qi X, Wei B, Carlini N, Huang Y, Xie T, He L, Jagielski M, Nasr M, Mittal P, Henderson P (2024) On evaluating the durability of safeguards for open-weight llms. *arXiv preprint arXiv:2412.07097*
- Qi X, Zeng Y, Xie T, Chen P-Y, Jia R, Mittal P, Henderson P (2024) Fine-tuning aligned language models compromises safety, even when users do not intend to! In: *12th Int. Conf. Learn. Represent. (ICLR 2024)*
- Rahman MA, Alqahtani L, Albooq A, Ainousah A (2024) A survey on security and privacy of large multi-modal deep learning models: Teaching and learning perspective. In: *21st Learn. Technol. Conf. (L & T 2024)*, pp. 13–18. IEEE, ???
- Rai P, Sood S, Madiseti VK, Bahga A (2024) Guardian: a multi-tiered defense architecture for thwarting prompt injection attacks on llms. *J Softw Eng Appl* 17(1):43–68
- Rajpal S (2023) Guardrails AI
- Rame A, Couairon G, Dancette C, Gaya J-B, Shukor M, Soulier L, Cord M (2023) Rewarded soups: towards pareto-optimal alignment by interpolating weights fine-tuned on diverse rewards. *Adv Neural Inf Process Syst* 36:71095–71134
- Ramezani A, Xu Y (2023) Knowledge of cultural moral norms in large language models. *arXiv preprint arxiv:2306.01857*
- Ranaldi L, Ruzzetti ES, Venditti D, Onorati D, Zanzotto FM (2023) A trip towards fairness: Bias and debiasing in large language models. *arXiv preprint arxiv:2305.13862*
- Rebedea T, Dinu R, Sreedhar M, Parisien C, Cohen J (2023) Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails. *arXiv preprint arxiv:2310.10501*
- Ren AZ, Dixit A, Bodrova A, Singh S, Tu S, Brown N, Xu P, Takayama L, Xia F, Varley J, et al (2023) Robots that ask for help: Uncertainty alignment for large language model planners. In: *2023 Conf. Robot Learn.*, pp. 661–682. PMLR, ???
- Ren J, Luo J, Zhao Y, Krishna K, Saleh M, Lakshminarayanan B, Liu PJ (2023) Out-of-distribution detection and selective generation for conditional language models. In: *11th Int. Conf. Learn. Represent. (ICLR 2023)*
- Robey A, Wong E, Hassani H, Pappas GJ (2023) Smoothllm: Defending large language models against jail-breaking attacks. *arXiv preprint arxiv:2310.03684*

- Rosenblatt L, Piedras L, Wilkins J (2022) Critical perspectives: A benchmark revealing pitfalls in PerspectiveAPI. In: Biester, L., Demszyk, D., Jin, Z., Sachan, M., Tetreault, J., Wilson, S., Xiao, L., Zhao, J. (eds.) Proc. 2nd Workshop NLP Posit. Impact (NLP4PI), pp. 15–24. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (Hybrid) . <https://doi.org/10.18653/v1/2022.nlp4pi-1.2>
- Röttger P, Kirk HR, Vidgen B, Attanasio G, Bianchi F, Hovy D (2023) Xstest: A test suite for identifying exaggerated safety behaviours in large language models. arXiv prepr. [arxiv:2308.01263](https://arxiv.org/abs/2308.01263)
- Ruan Y, Dong H, Wang A, Pitis S, Zhou Y, Ba J, Dubois Y, Maddison CJ, Hashimoto T (2023) Identifying the risks of lm agents with an lm-emulated sandbox. arXiv prepr. [arxiv:2309.15817](https://arxiv.org/abs/2309.15817)
- Sap M, Gabriel S, Qin L, Jurafsky D, Smith NA, Choi Y (2020) Social bias frames: Reasoning about social and power implications of language. In: Jurafsky, D., Chai, J., Schluter, N., Tetreault, J. (eds.) Proc. 58th Annu. Meet. Assoc. Comput. Linguist., pp. 5477–5490. Association for Computational Linguistics, Online . <https://doi.org/10.18653/v1/2020.acl-main.486>
- Sarker IH (2024) LLM potentiality and awareness: A position paper from the perspective of trustworthy and responsible AI modeling. Authorea Prepr
- Schwartz R, Vassilev A, Greene K, Perine L, Burt A, Hall P (2022) Towards a standard for identifying and managing bias in artificial intelligence. Nat Inst Standards Technol Gaithersburg MD. <https://doi.org/10.6028/NIST.SP.1270>
- Schwinn L, Dobre D, Khonneux S, Gidel G, Günnemann S (2024) Soft prompt threats: attacking safety alignment and unlearning in open-source llms through the embedding space. Adv Neural Inf Process Syst 37:9086–9116
- Shah MA, Sharma R, Dharmyal H, Olivier R, Shah A, Konan J, Alharthi D, Bukhari HT, Baali M, Deshmukh S, Kuhlmann M, Raj B, Singh R (2023) LoFT: Local proxy fine-tuning for improving transferability of adversarial attacks against large language model. arXiv prepr. [arxiv:2310.04445v2](https://arxiv.org/abs/2310.04445v2) [cs.CL]
- Shaikh O, Zhang H, Held W, Bernstein M, Yang D (2022) On second thought, let's not think step by step! Bias and toxicity in zero-shot reasoning. arXiv prepr. [arxiv:2212.08061](https://arxiv.org/abs/2212.08061)
- Shayegani E, Mamun MAA, Fu Y, Zaree P, Dong Y, Abu-Ghazaleh N (2023) Survey of vulnerabilities in large language models revealed by adversarial attacks. arXiv prepr. [arxiv:2310.10844](https://arxiv.org/abs/2310.10844)
- Shen X, Chen Z, Backes M, Shen Y, Zhang Y (2023) "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. arXiv prepr. [arxiv:2308.03825](https://arxiv.org/abs/2308.03825)
- Sheng Y, Cao S, Li D, Zhu B, Li Z, Zhuo D, Gonzalez JE, Stoica I (2023) Fairness in serving large language models. arXiv prepr. [arxiv:2401.00588](https://arxiv.org/abs/2401.00588)
- Sheppard B, Richter A, Cohen A, Smith EA, Kneese T, Pelletier C, Baldini I, Dong Y (2023) Subtle misogyny detection and mitigation: An expert-annotated dataset. arXiv prepr. [arxiv:2311.09443](https://arxiv.org/abs/2311.09443)
- Shi J, Liu Y, Zhou P, Sun L (2023) Badgpt: Exploring security vulnerabilities of chatgpt via backdoor attacks to instructgpt. arXiv prepr. [arxiv:2304.12298](https://arxiv.org/abs/2304.12298)
- Shi W, Shea R, Chen S, Zhang C, Jia R, Yu Z (2022) Just fine-tune twice: Selective differential privacy for large language models. arXiv prepr. [arxiv:2204.07667](https://arxiv.org/abs/2204.07667)
- Shuster K, Poff S, Chen M, Kiela D, Weston J (2021) Retrieval augmentation reduces hallucination in conversation. arXiv prepr. [arxiv:2104.07567](https://arxiv.org/abs/2104.07567)
- Shu M, Wang J, Zhu C, Geiping J, Xiao C, Goldstein T (2024) On the exploitability of instruction tuning. NeurIPS 36
- Simon N, Muise C (2022) TattleTale: Storytelling with planning and large language models. In: ICAPS Workshop Sched. Plan. Appl
- Singh S (2024) Flipkart Enhances AI Safety in E-Commerce: Implementing NVIDIA NeMo Guardrails. <https://blog.flipkart.tech/flipkart-enhances-ai-safety-in-e-commerce-implementing-nvidia-nemo-guardrails-cb2f293b29c0>
- Singhal K, Tu T, Gottweis J, Sayres R, Wulczyn E, Hou L, Clark K, Pfohl S, Cole-Lewis H, Neal D, et al (2023) Towards expert-level medical question answering with large language models. arXiv prepr. [arxiv:2305.09617](https://arxiv.org/abs/2305.09617)
- Song L, Shokri R, Mittal P (2019) Privacy risks of securing machine learning models against adversarial examples. In: Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur., pp. 241–257. Association for Computing Machinery, London, United Kingdom
- Sun H, Pei J, Choi M, Jurgens D (2023) Aligning with whom? large language models have gender and racial biases in subjective nlp tasks. arXiv prepr. [arxiv:2311.09730](https://arxiv.org/abs/2311.09730)
- Tamirisa R, Bharathi B, Phan L, Zhou A, Gatti A, Suresh T, Lin M, Wang J, Wang R, Arel R, et al (2024) Tamper-resistant safeguards for open-weight llms. arXiv preprint [arXiv:2408.00761](https://arxiv.org/abs/2408.00761)
- Tang X, Jin Q, Zhu K, Yuan T, Zhang Y, Zhou W, Qu M, Zhao Y, Tang J, Zhang Z, et al (2024) Prioritizing safeguarding over autonomy: risks of LLM agents for science. arXiv prepr. [arxiv:2402.04247](https://arxiv.org/abs/2402.04247)
- Tao Y, Viberg O, Baker RS, Kizilcec RF (2023) Auditing and mitigating cultural bias in LLMs. arXiv prepr. [arxiv:2311.14096](https://arxiv.org/abs/2311.14096)
- team GA (2023) Tutorial of Guidance AI. <https://guidance.readthedocs.io/en/latest/tutorials.html>

- Team GA (2024) use cases of Guardrails AI. <https://hub.guardrailsai.com>
- Team N (2024a) Building Blocks for Agentic AI. <https://www.nvidia.com/en-gb/ai/>
- Team T (2023) Documentation of Guardrails AI. <https://www.trulens.org>
- Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M-A, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, et al (2023) Llama: Open and efficient foundation language models. arXiv prepr. [arxiv:2302.13971](https://arxiv.org/abs/2302.13971)
- Trist EL, Bamforth KW (1957) Studies in the Quality of Life: Delivered by the Institute of Personnel Management in November 1957. Lecture Series
- Ungless EL, Rafferty A, Nag H, Ross B (2022) A Robust Bias Mitigation procedure based on the stereotype content model. arXiv prepr. [arxiv:2210.14552](https://arxiv.org/abs/2210.14552)
- van Lamsweerde A, Darimont R, Letier E (1998) Managing conflicts in goal-driven requirements engineering. *IEEE Trans Softw Eng* 24(11):908–926. <https://doi.org/10.1109/32.730542>
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) *Adv. Neural Inf. Process. Syst.* 30 (NeurIPS 2017), vol. 30. Curran Associates, Inc., ???
- Vega J, Chaudhary I, Xu C, Singh G (2023) Bypassing the safety training of open-source llms with priming attacks. arXiv preprint [arXiv:2312.12321](https://arxiv.org/abs/2312.12321)
- Vinyals O, Fortunato M, Jaitly N (2015) Pointer networks. In: Cortes, C., Lawrence, N., Lee, D., Sugiyama, M., Garnett, R. (eds.) *Adv. Neural Inf. Process. Syst.* 28 (NeurIPS 2015), vol. 28. Curran Associates, Inc., ???
- Vivien (2023) Better Steering LLM Agents with LMQL. <https://vivien000.github.io/blog/journal/better-steering-LLM-agents-with-LMQL.html>
- Wachi A, Tran T, Sato R, Tanabe T, Akimoto Y (2024) Stepwise alignment for constrained language model policy optimization. *Adv Neural Inf Process Syst* 37:104471–104520
- Wang B, Chen W, Pei H, Xie C, Kang M, Zhang C, Xu C, Xiong Z, Dutta R, Schaeffer R et al (2023) Decodingtrust: a comprehensive assessment of trustworthiness in gpt models. *Adv Neural Inf Process Syst* 36:31232–31339
- Wang B, Chen W, Pei H, Xie C, Kang M, Zhang C, X, C, Xiong Z, Dutta R, Schaeffer R, Truong ST, Arora S, Mazeika M, Hendrycks D, Lin Z, Cheng Y, Koyejo S, Song D, Li B (2024) DecodingTrust: A comprehensive assessment of trustworthiness in GPT models. arXiv prepr. [arxiv:2306.11698](https://arxiv.org/abs/2306.11698)
- Wang Y, Liu X, Li Y, Chen M, Xiao C (2024) Adashield: Safeguarding multimodal large language models from structure-based attack via adaptive shield prompting. In: *European Conference on Computer Vision*, pp. 77–94. Springer
- Wang H, Shu K (2023) Backdoor activation attack: Attack large language models using activation steering for safety-alignment. arXiv prepr. [arxiv:2311.09433](https://arxiv.org/abs/2311.09433)
- Wang X, Wang H, Yang D (2022) Measure and improve robustness in NLP models: A survey. arXiv prepr. [arxiv:2112.08313v2](https://arxiv.org/abs/2112.08313v2) [cs.CL]
- Wang X, Wei J, Schuurmans D, Le QV, Chi EH, Narang S, Chowdhery A, Zhou D (2023) Self-consistency improves chain of thought reasoning in language models. In: *11th Int. Conf. Learn. Represent. (ICLR 2023)*
- Wan S, Nikolaidis C, Song D, Molnar D, Crnkovich J, Grace J, Bhatt M, Chennabasappa S, Whitman S, Ding S, et al (2024) Cyberseceval 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models. arXiv preprint [arXiv:2408.01605](https://arxiv.org/abs/2408.01605)
- Webster M, Schmitt J (2024) LLM hallucinations: How to detect and prevent them with CI. *CircleCI Blog* (Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS* 35:24824–24837
- Wei A, Haghtalab N, Steinhart J (2024) Jailbroken: how does llm safety training fail? *NeurIPS* 36:119
- Wei J, Kim S, Jung H, Kim Y-H (2023) Leveraging large language models to power chatbots for collecting user self-reported data. arXiv prepr. [arxiv:2301.05843](https://arxiv.org/abs/2301.05843)
- Wei Z, Wang Y, Wang Y (2023) Jailbreak and guard aligned language models with only few in-context demonstrations. arXiv prepr. [arxiv:2310.06387](https://arxiv.org/abs/2310.06387)
- Welbl J, Glaese A, Uesato J, Dathathri S, Mellor J, Hendricks LA, Anderson K, Kohli P, Coppin B, Huang P-S (2021) Challenges in detoxifying language models. arXiv prepr. [arxiv:2109.07445](https://arxiv.org/abs/2109.07445)
- Welbl J, Glaese A, Uesato J, Dathathri S, Mellor J, Hendricks LA, Anderson K, Kohli P, Coppin B, Huang P-S (2021) Challenges in detoxifying language models. In: Moens, M.-F., Huang, X., Specia, L., Yih, S.W.-t. (eds.) *Find. Assoc. Comput. Linguist.: EMNLP 2021*, pp. 2447–2469. Association for Computational Linguistics, ???
- Weng F, Xu Y, Fu C, Wang W (2024) Mmj-bench: A comprehensive study on jailbreak attacks and defenses for multimodal large language models. arXiv preprint [arXiv:2408.08464](https://arxiv.org/abs/2408.08464)
- Wen J, Ke P, Sun H, Zhang Z, Li C, Bai J, Huang M (2023) Unveiling the implicit toxicity in large language models. In: *2023 Conf. Empir. Methods Nat. Lang. Process. (EMNLP 2023)*

- Xiang A (2022) Being “seen” vs. “mis-seen”: tensions between privacy and fairness in computer vision. *Harv J Law Technol* 36:1
- Xiao Y, Jin Y, Bai Y, Wu Y, Yang X, Luo X, Yu W, Zhao X, Liu Y, Chen H, et al (2023) Large language models can be good privacy protection learners. arXiv prepr. [arxiv:2310.02469](https://arxiv.org/abs/2310.02469)
- Xiao Y, Liang PP, Bhatt U, Neiswanger W, Salakhutdinov R, Morency L-P (2022) Uncertainty quantification with pre-trained language models: A large-scale empirical analysis. arXiv prepr. [arxiv:2210.04714](https://arxiv.org/abs/2210.04714)
- Xie Y, Yi J, Shao J, Curl J, Lyu L, Chen Q, Xie X, Wu F (2023) Defending chatgpt against jailbreak attack via self-reminders. *Nat Mach Intell* 5(12):1486–1496
- Xie Z, Lukaszewicz T (2023) An empirical analysis of parameter-efficient methods for debiasing pre-trained language models. In: Rogers, A., Boyd-Graber, J.L., Okazaki, N. (eds.) *Proc. 61st Annu. Meet. Assoc. Comput. Linguist.*, pp. 15730–15745. Association for Computational Linguistics, ??? <https://doi.org/10.18653/v1/2023.ACL-LONG.876>
- Xu Z, Jiang F, Niu L, Jia J, Lin BY, Poovendran R (2024) SafeDecoding: Defending against jailbreak attacks via safety-aware decoding. arXiv prepr. [arxiv:2402.08983](https://arxiv.org/abs/2402.08983)
- Yang Y, Dan S, Roth D, Lee I (2024) Benchmarking llm guardrails in handling multilingual toxicity. arXiv preprint [arXiv:2410.22153](https://arxiv.org/abs/2410.22153)
- Yang R, Fu M, Tantithamthavorn C, Arora C, Vandenhurk L, Chua J (2025) Ragva: Engineering retrieval augmented generation-based virtual assistants in practice. *Journal of Systems and Software*, 112436
- Yang J, Zhang X, Liang K, Liu Y (2023) Exploring the application of large language models in detecting and protecting personally identifiable information in archival data: a comprehensive study. In: *IEEE Int. Conf. Big Data (BigData)*, pp. 2116–2123. IEEE, ???
- Yao H, Lou J, Ren K, Qin Z (2023) PromptCARE: Prompt copyright protection by watermark injection and verification. In: *2024 IEEE Symp. Secur. Priv. (SP 2024)*
- Yeh K-C, Chi J-A, Lian D-C, Hsieh S-K (2023) Evaluating interfaced LLM bias. In: *Proc. 35th Conf. Comput. Linguist. Speech Process. (ROCLING 2023)*, pp. 292–299
- Ye W, Ou M, Li T, chen Y, Ma X, Yanggong Y, Wu S, Fu J, Chen G, Wang H, Zhao J (2023) Assessing hidden risks of LLMs: An empirical study on robustness, consistency, and credibility. arXiv prepr. [arxiv:2305.10235v4](https://arxiv.org/abs/2305.10235v4) [cs.LG]
- Yi S, Liu Y, Sun Z, Cong T, He X, Song J, Xu K, Li Q (2024) Jailbreak attacks and defenses against large language models: A survey. arXiv preprint [arXiv:2407.04295](https://arxiv.org/abs/2407.04295)
- Yin X, Qi Y, Hu J, Chen Z, Dong Y, Zhao X, Huang X, Ruan W (2025) Taiji: Textual anchoring for immunizing jailbreak images in vision language models. arXiv preprint [arXiv:2503.10872](https://arxiv.org/abs/2503.10872)
- Yong ZX, Menghini C, Bach S (2023) Low-resource languages jailbreak GPT-4. In: *Soc. Responsible Lang. Model. Res*
- Yu X (2024) Create an AI Agent with Llama Guard in Anypoint Platform. <https://medium.com/@yuxiaojian/create-an-ai-agent-with-llama-guard-in-anypoint-platform-a313b2c0b51f>
- Yuan T, He Z, Dong L, Wang Y, Zhao R, Xia T, Xu L, Zhou B, Li F, Zhang Z, et al (2024) R-judge: Benchmarking safety risk awareness for LLM agents. arXiv prepr. [arxiv:2401.10019](https://arxiv.org/abs/2401.10019)
- Yuan Y, Jiao W, Wang W, Huang J-t, He P, Shi S, Tu Z (2024) GPT-4 is too smart to be safe: Stealthy chat with LLMs via cipher. In: *12th Int. Conf. Learn. Represent. (ICLR 2024)*
- Yu J, Lin X, Xing X (2023) Gptfuzzer: Red teaming large language models with auto-generated jailbreak prompts. arXiv prepr. [arxiv:2309.10253](https://arxiv.org/abs/2309.10253)
- Yu D, Naik S, Backurs A, Gopi S, Inan HA, Kamath G, Kulkarni J, Lee YT, Manoel A, Wutschitz L, Yekhanin S, Zhang H (2022) Differentially private fine-tuning of language models. In: *10th Int. Conf. Learn. Represent. (ICLR 2022)*. OpenReview.net
- Zampieri M, Malmasi S, Nakov P, Rosenthal S, Farra N, Kumar R (2019) Predicting the type and target of offensive posts in social media. In: Burstein, J., Doran, C., Solorio, T. (eds.) *Proc. 2019 Conf. N. Am. Chapter Assoc. Comput. Linguist.: Hum. Lang. Technol.*, pp. 1415–1420. Association for Computational Linguistics, Minneapolis, Minnesota. <https://doi.org/10.18653/v1/N19-1144>
- Zeng C, Li S, Li Q, Hu J, Hu J (2020) A survey on machine reading comprehension-tasks, evaluation metrics and benchmark datasets. *Appl Sci* 10(21):7640
- Zhan Q, Fang R, Bindu R, Gupta A, Hashimoto T, Kang D (2023) Removing rlhf protections in gpt-4 via fine-tuning. arXiv prepr. [arxiv:2311.05553](https://arxiv.org/abs/2311.05553)
- Zhang W, Deng Y, Liu B, Pan SJ, Bing L (2023) Sentiment analysis in the era of large language models: A reality check. arXiv prepr. [arxiv:2305.15005](https://arxiv.org/abs/2305.15005)
- Zhang H, Guo Z, Zhu H, Cao B, Lin L, Jia J, Chen J, Wu D (2023) On the safety of open-sourced large language models: Does alignment really prevent them from being misused? arXiv prepr. [arxiv:2310.01581](https://arxiv.org/abs/2310.01581)
- Zhang B, Shen X, Si WM, Sha Z, Chen Z, Salem A, Shen Y, Backes M, Zhang Y (2023) Comprehensive assessment of toxicity in ChatGPT. arXiv prepr. [arxiv:2311.14685](https://arxiv.org/abs/2311.14685) [cs.CY]
- Zhang Z, Yang J, Ke P, Huang M (2023) Defending large language models against jailbreaking attacks through goal prioritization. arXiv prepr. [arxiv:2311.09096](https://arxiv.org/abs/2311.09096)

- Zhao J, Chen K, Yuan X, Qi Y, Zhang W, Yu N (2023) Silent guardian: Protecting text from malicious exploitation by large language models. arXiv prepr. [arxiv:2312.09669](https://arxiv.org/abs/2312.09669)
- Zhao S, Jia M, Tuan LA, Wen J (2024) Universal vulnerabilities in large language models: In-context learning backdoor attacks. arXiv prepr. [arxiv:2401.05949](https://arxiv.org/abs/2401.05949)
- Zhou KZ, Sanfilippo MR (2023) Public perceptions of gender bias in large language models: Cases of chatgpt and ernie. arXiv prepr. [arxiv:2309.09120](https://arxiv.org/abs/2309.09120)
- Zhou A, Li B, Wang H (2024) Robust prompt optimization for defending language models against jailbreaking attacks. arXiv prepr. [arxiv:2401.17263](https://arxiv.org/abs/2401.17263)
- Zhou W, Wang X, Xiong L, Xia H, Gu Y, Chai M, Zhu F, Huang C, Dou S, Xi Z, et al (2024) EasyJailbreak: A unified framework for jailbreaking large language models. arXiv prepr. [arxiv:2403.12171](https://arxiv.org/abs/2403.12171)
- Zhu K, Wang J, Zhou J, Wang Z, Chen H, Wang Y, Yang L, Ye W, Zhang Y, Gong NZ, Xie X (2023) Prompt-Bench: Towards evaluating the robustness of large language models on adversarial prompts. arXiv prepr. [arxiv:2306.04528v4](https://arxiv.org/abs/2306.04528v4) [cs.CL]
- Zhu S, Zhang R, An B, Wu G, Barrow J, Wang Z, Huang F, Nenkova A, Sun T (2023) Autodan: Automatic and interpretable adversarial attacks on large language models. arXiv prepr. [arxiv:2310.15140](https://arxiv.org/abs/2310.15140)
- Zou W, Geng R, Wang B, Jia J (2024) PoisonedRAG: Knowledge poisoning attacks to retrieval-augmented generation of large language models. arXiv prepr. [arxiv:2402.07867](https://arxiv.org/abs/2402.07867)
- Zou A, Wang Z, Carlini N, Nasr M, Kolter JZ, Fredrikson M (2023) Universal and transferable adversarial attacks on aligned language models. arXiv preprint [arXiv:2307.15043](https://arxiv.org/abs/2307.15043)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.